

In [1]:

```
int('111',2)
```

Out[1]:

7

In []:

```
class Solution:
    def groupAnagrams(self, strs):
        l=[]
        for i in strs:
            l+=[''.join(sorted(i))]
        res=[]
        i=0
        while(i<len(l)):
            item=l[0]
            obj=Solution()
            obj.groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
```

In []:

In [1]:

```
from itertools import permutations
class Solution:
    def getPermutation(self, n , k ):
        m=[i for i in range(1,n+1)]
        a=list(permutations(m))
        res=''.join(map(str,a[k-1]))
        return res
obj=Solution()
obj.getPermutation(5,1)
```

Out[1]:

'12345'

In [7]:

```
from itertools import *
a=[1,2,3]
print(list(permutations(a,2)))
print(list(combinations(a,2)))

[(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
[(1, 2), (1, 3), (2, 3)]
```

In [68]:

```
a=[1,2,3,4]
''.join(map(str,a))
```

Out[68]:

```
'1234'
```

In [57]:

```
from itertools import permutations
class Solution:
    def permute(self, nums):
        a=permutations(nums)
        org=[]
        for i in list(a):
            b=list(i)
            org+=b
        return org
obj=Solution()
obj.permute([1,2,3])
```

Out[57]:

```
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

In [71]:

```
from itertools import permutations
class Solution:
    def findEvenNumbers(self, digits):
        a=permutations(digits,3)
        org=[]
        for i in a:
            org+= [int(''.join(map(str,i)))]
        res=[]
        for i in org:
            if (i>99 and i not in res and i%2==0):
                res+= [i]
        return res
obj=Solution()
obj.findEvenNumbers([2,1,3,0])
```

Out[71]:

```
[210, 230, 120, 132, 130, 102, 320, 312, 310, 302]
```

In []:

```
a="2"
print(type(a))
b="3"
print(type(chr(ord(a))))
print(ord(b))
print(str(eval(a+'*'+b)))
```

In []:

```

class Solution:
    def searchRange(self, nums , target ):
        if target not in nums:
            b=[-1,-1]
            return b
        else:
            b=[nums.index(target),len(nums)-1-nums[::-1].index(target)]
            return b
obj=Solution()
obj.searchRange([0,0,0,0,1,2,3,3,4,5,6,6,7,8,8,8,9,9,10,10,11,11],0)

```

In []:

```

class Solution():
    val=0
    def addTwoNumbers(self,l1=[],l2=[]):
        self.l1=l1
        self.l2=l2
        s1=0
        res=[]
        s2=0
        l1=l1[::-1]
        l2=l2[::-1]
        for i in range(len(l1)):
            s1+=(10**i)*l1[i]
        for j in range(len(l2)):
            s2+=(10**j)*l2[j]
        self.val=s1+s2
        org=self.val
        for i in range(len(str(org))):
            rem=org%10
            res+=[rem]
            org=org//10
        print(res)
obj=Solution()
obj.addTwoNumbers([9,9,9,9,9,9,9],[9,9,9,9])

```

In []:

```

class Solution(object):
    def addTwoNumbers(self, list(11), list(12)):
        l1.reverse()
        l2.reverse()
        a,b=0,0
        for i in range(len(l1)):
            x=l1[i]
            y=l2[i]
            a+=(10**i*x)
            b+=(10**i*y)
        res=a+b
        rem=0
        r1=[]
        for i in range(len(str(res))):
            rem=res%10
            res=res//10
            r1.insert(0,rem)
        print(r1[::-1])
obj=Solution()
obj.addTwoNumbers([2,4,3],[5,6,4])

```

In []:

```

class Solution():
    def findMedianSortedArrays(self, nums1, nums2):
        self.nums1=nums1
        self.nums2=nums2
        nums1+=nums2
        nums1.sort()
        l=len(nums1)
        median=0
        if(l%2==0):
            value=nums1[l//2]+nums1[(l//2)-1]
            median =(value/2)
            return median
        else:
            median=nums1[l//2]
            return median
obj=Solution()
obj.findMedianSortedArrays([1,2],[3,4])

```

In []:

```
2**31-1
```

In []:

```

x=-123
x=int(str(abs(x))[::-1])
x

```


In []:

```

class Solution(object):
    def reverse(self, x):
        if(x>=(2**16)-1 | x<=(-1*2**16)):
            return 0
        else:
            if(x<0):
                y=int(str(abs(x))[::-1])
                return y*-1
            else:
                y=int(str(abs(x))[::-1])
                return y
ob=Solution()
ob.reverse(1534236469)

```

In []:

```
2**31-1>1534236469
```

In []:

```
2**30-1>1463847412
```

In []:

```
1463847412
```

In []:

```
"a"+"b"
```

In []:

```

class Solution(object):
    def lengthOfLongestSubstring(self, s):
        l=[]
        if (len(s)<=5*10**4):
            for i in range(len(s)+1):
                sub=""
                for j in s[i:]:
                    if j in sub:
                        l+= [len(sub)]
                        sub=""
                    else:
                        sub+=j
                l+= [len(sub)]
            return max(l)
obj=Solution()
obj.lengthOfLongestSubstring("pwwkew")

```

In []:

```
len("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!\"#$%&'()*+,-./:;<=>?@[
```

In []:

```
l=[1,2,3,4]
l.pop(1)
l.pop(2)
l
```

In []:

```
a="abc"
a=list(a)
b=''.join(a)
b
```

In []:

```
x=2
i=0
while(i<=2):
    print(i)
    i+=1
    if(i==1):
        i=0
        x=5
```

In []:

```
"""class Solution(object):
    def removeDuplicates(self, s):
        i=0
        l=list(s)
        while(i!=len(l)-1):
            if(l[i]==l[i+1]):
                l.pop(i)
                l.pop(i)
                i=i-1
                if(i<0):
                    i=0
            else:
                i+=1
        val=''.join(l)
        return val
obj=Solution()
obj.removeDuplicates("aaaaaaa")"""
```

In []:

```
a=[-1,-2,-3,-5]
maxi=abs(max(a))
maxi
```

In []:

```
2**31 - 1 > 2147483647
```

In []:

```

class Solution(object):
    def trap(self, height):
        dup=[]
        for i in height:
            if i not in dup:
                dup+= [i]
        for i in dup:
            ind=height.index("i")
            for j in range(ind,len(height)):
                i

```

In [13]:

```

a=["flower","flow","flight","", "a"]
b=sorted(a,key=len)
b

```

Out[13]:

```
['', 'a', 'flow', 'flower', 'flight']
```

In [8]:

```

from itertools import *
a=[1,2,3]
print(list(permutations(a,2)))
print(list(combinations(a,2)))

[(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
[(1, 2), (1, 3), (2, 3)]

```

In []:

```

class Solution(object):
    def longestCommonPrefix(self, strs):
        a=sorted(strs,key=len)
        l=len(a[0])
        m=len(a)
        org=""
        for i in range(l):
            comp=a[0][i]
            for j in a:
                if (j[i]!=comp):
                    return org
obj=Solution()
obj.longestCommonPrefix(["flower","flow","flight"])

```

In []:

```

a=["flower","flow","flight","", "a"]
a[0][0]

```

In []:

```
"flow" in "flower"
```

In []:

```

class Solution(object):
    def longestCommonPrefix(self, strs):
        self.strs=strs
        a=sorted(strs,key=len)
        b=a[0]
        c=""
        org=[]
        for i in range(len(a[0])):
            c+=b[i]
            org+=c
        org.insert(0,"")
        for i in org:
            for j in strs:
                if i not in j:
                    return (org[org.index(i)-1])
obj=Solution()
obj.longestCommonPrefix(["","a"])

```

In [14]:

```
"" in "b"
```

Out[14]:

True

In []:

```

class Solution(object):
    def longestCommonPrefix(self, strs):
        a=sorted(strs,key=len)
        b=a[0]
        c=""
        org=[]
        if len(strs)==0:
            return ""
        else:
            for i in range(len(a[0])):
                c+=b[i]
                org+=c
            org.insert(0,"")
            for i in org:
                for j in strs:
                    if i not in j:
                        a=(org[org.index(i)])
                        if a==org[0]:
                            print("aaa")
                            return org[0]
                        else:
                            return (org[org.index(i)-1])
obj=Solution()
obj.longestCommonPrefix([""])

```

In []:

```
class Solution:
    def isValid(self, str):
        ob=["(", "[", "{"]
        cb="){","}]","}"
        ol=[]
        cl=[]
        for i in str:
            elif i in ob:
                ind=ob.index(i)
                ol+=i
                cl.insert(0,cb[ind])
        return ol
obj=Solution()
obj.isValid("({[]})")
```

In [2]:

```
a=[1,2,3]
res=''.join(map(str,a))
res
b=int(res)+1
list(map(int,str(b)))
```

Out[2]:

[1, 2, 4]

In [27]:

```
n=4
print('{0:b}'.format(4))
bin(8)[2:]
```

100

Out[27]:

'1000'

In [28]:

```
min(22,30,2)
```

Out[28]:

2

In [29]:

```
a=[1,2,3,4]
max(2,max(a))
```

Out[29]:

4

In []:

```

class Solution:
    def largestRectangleArea(self, heights):
        if(len(heights)==0):
            return 0
        else:
            m=max(heights)
            minl=[]
            for i in range(len(heights)-1):
                minl+= [min(heights[i],heights[i+1])*2]
            return (max(m,max(minl)))
obj=Solution()
obj.largestRectangleArea([])

```

In []:

```

l=[]
for i in range(5):
    dvn="p"+str(i)
    l+= [dvn]
l

```

In []:

```

a={}
type(a)
chr(65+j)

```

In []:

```

nor=5
for j in range(nor):
    p=chr(65+j)
    val=int(input("Enter allocated resource of"+p))
    val

```

In []:

```

class Solution:
    def longestPalindrome(self, s ):
        if len(s)==1:
            return s
        else:
            l=[]
            for i in range(len(s)-1):
                for j in range(i+1,len(s)):
                    org=s[i:j+1]
                    if(org==org[::-1]):
                        l+= [org]
            return max(l,key=len)
obj=Solution()
obj.longestPalindrome("bb")

```

In []:

```

a=['_']
a

```

In []:

```
import re
class Solution:
    def myAtoi(self,s):
        return(int(re.search(r'[-?]\d+',s).group()))
obj=Solution()
obj.myAtoi("words and 987")
```

In []:

```
class Solution:
    def isPalindrome(self, x):
        if(str(x)==str(x)[::-1]):
            return 'true'
        else:
            return 'false'
obj=Solution()
obj.isPalindrome(-121)
```

In []:

```
val={"(":1,"[":5,"{":10,"L":50,"C":100,"D":500,"M":1000}
val
```

In []:

```
strs = ["flower","flow","flight"]
a=sorted(strs,key=len)[0]
strs.remove(a)
len(strs)
```

In []:

```
class Solution:
    def divide(self, dividend , divisor):
        num=0
        if (divisor<0 and divisor>0) or (divisor>0 and divisor<0):
            num=1
            divisor=abs(divisor)
            dividend=abs(dividend)
        count=0
        while(dividend>=divisor):
            count+=1
            dividend-=divisor
        if num==1:
            return -count
        else:
            return count
obj=Solution()
obj.divide(-1,1)
```

In []:

```
class Solution:
    def searchInsert(self, nums , target):
        if target in nums:
            return nums.index(target)
        else:
            for i in range(len(nums)):
                if target <= nums[i]:
                    return i
            break
obj=Solution()
obj.searchInsert([1,3,6],5)
```

In []:

```
class Solution:
    def isPalindrome(self, s):
        org=""
        for i in s:
            if i.isalnum():
                org+=i
        l=""
        for i in org:
            if (ord(i)>=65 and ord(i)<=90):
                l+=(chr(ord(i)+32))
            else:
                l+=i
        if(l==l[::-1]):
            return "true"
        else:
            return "false"
obj=Solution()
obj.isPalindrome("race a car")
```

In []:

```
class Solution:
    def reverseWords(self, s):
        li=s.split(" ")
        li=li[::-1]
        li=list(filter((').__ne__',li))
        print(li)
        return (' '.join(li))
obj=Solution()
obj.reverseWords(" hello world ")
```


In []:

```
class Solution:
    def isValid(self, s):
        ob=["(", "[", "{"]
        cb="){","}]","}"
        l=[]
        if s[0] in cb:
            return "false"
        for i in s:
            if i in ob:
                l+=i
            else:
                ind=cb.index(i)
                if ob[ind]==l[len(l)-1]:
                    l.pop()
                else:
                    return "false"
        if (len(l)==0):
            return "true"
        else:
            return "false"
obj=Solution()
obj.isValid("({}[[]])")
```

In []:

```
class Solution:
    def removeDuplicates(self, nums):
        org=[]
        un=[]
        for i in nums:
            if i not in org:
                org.append(i)
            else:
                un.append('_')
        print(org)
obj=Solution()
obj.removeDuplicates([1,1,2])
```

In []:

```
a=[1,2,3,4]
a
```

In []:

```

class Solution:
    # @param A : tuple of integers
    # @return an integer
    def longestConsecutive(self, A):
        self.A=A
        if len(A)==0:
            return 0
        else:
            A=list(A)
            A.sort()
            print(A)
            org=[]
            org.append(A[0])
            se=org[0]
            print(se)
            for i in range(1,len(A)):
                print(A[i])
                print
                if (A[i]-se==1):
                    org.append(A[i])
                    se=A[i]
                    print(se)
                    print(org)
                else:
                    break
            break
            return(len(org))
obj=Solution()
obj.longestConsecutive((100,4,200,1,2,3))

```

In []:

```

class Solution:
    def maxSubArray(self, nums):
        nums=sorted(nums)
        p=[i for i in nums if i>=0]
        n=[i for i in nums if i<0]
        if len(p)==0:
            return n[0]
        else:
            return (sum(p))
obj=Solution()
obj.maxSubArray([-2,1,-3,4,-1,2,1,-5,4])

```

In []:

```

from itertools import permutations
class Solution:
    def getPermutation(self, n , k ):
        m=[i for i in range(1,n+1)]
        a=list(permutations(m))
        print(a)
        res=''.join(map(str,a[k-1]))
        return res
obj=Solution()
obj.getPermutation(3,3)

```

In []:

```
class Solution:
    def setZeroes(self, matrix):
        rows=[]
        cols=[]
        for i in range(len(matrix)):
            for j in range(len(matrix[i])):
                if (matrix[i][j]==0):
                    if i not in rows:
                        rows+= [i]
                    if j not in cols:
                        cols+= [j]
        for i in range(len(rows)):
            for j in range(len(matrix[0])):
                matrix[rows[i]][j]=0
        for i in range(len(matrix)):
            for j in range(len(cols)):
                matrix[i][cols[j]]=0
        print(matrix)
obj=Solution()
obj.setZeroes([[1,1,1],[1,0,1],[1,1,1]])
```

In [31]:

```
matrix=[[1,1,1,1],[1,0,1,0],[1,1,1,1]]
len(matrix)
```

Out[31]:

3

In []:

```
class Solution:
    def mySqrt(self, x):
        if (x==0 or x==1):
            return x
        else:
            val=4
            i=2
            while(val<=x):
                i=i+1
                val=i*i
            return (i-1)
obj=Solution()
obj.mySqrt(8)
```

In []:

```

class Solution:
    def convert(self, s, numRows):
        if (numRows==1 or len(s)==1):
            return s
        cols=int(len(s)*(numRows-1)/(2*numRows-2))+1
        m=[['' for i in range(cols)] for j in range(numRows)]
        i,j,k=0,0,0
        dir=1
        while(k<len(s)):
            if(i!=numRows-1 and dir==1):
                m[i][j]=s[k]
                i+=1
                k+=1
                if (i==numRows-1):
                    dir=0
            elif(i==numRows-1 or dir==0):
                m[i][j]=s[k]
                j+=1
                i-=1
                k+=1
                if (i==0):
                    dir=1
            val=""
            for i in range(numRows):
                for j in range(cols):
                    val+=m[i][j]
            return val
obj=Solution()
obj.convert("abc",2)

```

In []:

```

s="pay"
numRows=2
cols=int(len(s)*(numRows-1)/(2*numRows-2))+1
print(cols)
mat=[['' for i in range(cols)] for j in range(numRows)]
mat

```

In []:

```
'a'+' '+'b'
```

In []:

```

class Solution:
    def searchInsert(self, nums , target):
        if target in nums:
            return nums.index(target)
        else:
            for i in range(len(nums)):
                if (i==len(nums)-1):
                    return len(nums)
                elif target <= nums[i]:
                    return i
obj=Solution()
obj.searchInsert([1,3,5,6],7)

```

In []:

```

class Solution:
    def divide(self, dividend , divisor):
        num=0
        if (divisor<0 and dividend>0) or (divisor>0 and dividend<0):
            num=1
            divisor=abs(divisor)
            dividend=abs(dividend)
        count=0
        divisor=abs(divisor)
        dividend=abs(dividend)
        while(dividend>=divisor):
            count+=1
            dividend-=divisor
        if num==1:
            return -count
        else:
            return count
obj=Solution()
obj.divide(-1,-1)

```

In [39]:

```

res=''.join(map(str,[1,2,3]))
print(type(res))
a=int(res)
(list(map(int,'456'))))

```

<class 'str'>

Out[39]:

[4, 5, 6]

In []:

```

class Solution:
    def isPalindrome(self, s):
        org=""
        for i in s:
            if i.isalnum() is True:
                org+=i
        l=""
        for i in org:
            if (ord(i)>=65 and ord(i)<=90):
                l+=(chr(ord(i)+32))
            else:
                l+=i
        if(l==l[::-1]):
            return 'true'
        else:
            return 'false'
obj=Solution()
obj.isPalindrome("race a car")

```

In []:

```
class Solution:
    def removeDuplicates(self, nums):
        org=[]
        un=[]
        for i in nums:
            if i not in org:
                org.append(i)
        return org
obj=Solution()
obj.removeDuplicates([1,1,2])
```

In []:

```

def isvalid(board):
    def validate(a,i,j):
        l1=[]
        l2=[]
        s1=[0,1,2]
        s2=[3,4,5]
        s3=[6,7,8]
        if i in s1:
            l1=s1
            if j in s1:
                l2=s1
            elif j in s2:
                l2=s2
            else:
                l2=s3
        elif i in s2:
            l1=s2
            if j in s1:
                l2=s1
            elif j in s2:
                l2=s2
            else:
                l2=s3
        else:
            l1=s3
            if j in s1:
                l2=s1
            elif j in s2:
                l2=s2
            else:
                l2=s3
        count=0
        for x in l1:
            for y in l2:
                if board[x][y].isalnum():
                    if (int(board[x][y])==a):
                        count+=1
        for y in range(9):
            if board[i][y].isalnum():
                if (int(board[i][y])==a):
                    count+=1
        print(count)
        if (count>2):
            return "false"
    for i in range(9):
        for j in range(9):
            if board[i][j].isalnum():
                a=int(board[i][j])
                if validate(a,i,j=="false"):
                    return 'false'
    return "true"
boarddvd = [["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],["."
board=[["8","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9"
print(isvalid(board))

```

In [40]:

```
a="."  
x=a.isalnum()  
x
```

Out[40]:

False

In []:

```

class Solution:
    def isValidSudoku(self, board):
        def validate(a,i,j):
            l1=[]
            l2=[]
            s1=[0,1,2]
            s2=[3,4,5]
            s3=[6,7,8]
            if i in s1:
                l1=s1
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            elif i in s2:
                l1=s2
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            else:
                l1=s3
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            count=0
            for x in l1:
                for y in l2:
                    if board[x][y].isalnum():
                        if (int(board[x][y])==a):
                            count+=1
            for y in range(9):
                if board[i][y].isalnum():
                    if (int(board[i][y])==a):
                        count+=1
            if (count>2):
                return "false"
        for i in range(9):
            for j in range(9):
                if board[i][j].isalnum():
                    a=int(board[i][j])
                    if validate(a,i,j)=="false":
                        return 'false'
        return "true"
obj=Solution()
obj.isValidSudoku([[ "5", "3", ".", ".", "7", ".", ".", ".", "." ], [ "6", ".", ".", "1", "9", "5", ".", ".", "

```

In []:

```

class Solution:
    def isValidSudoku(self, board):
        def validate(a,i,j):
            l1=[]
            l2=[]
            s1=[0,1,2]
            s2=[3,4,5]
            s3=[6,7,8]
            if i in s1:
                l1=s1
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            elif i in s2:
                l1=s2
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            else:
                l1=s3
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            count=0
            for x in l1:
                for y in l2:
                    if board[x][y].isalnum():
                        if (int(board[x][y])==a):
                            count+=1
            for y in range(9):
                if board[i][y].isalnum():
                    if (int(board[i][y])==a):
                        count+=1
            if (count>2):
                return "false"
        for i in range(9):
            for j in range(9):
                if board[i][j].isalnum():
                    a=int(board[i][j])
                    if validate(a,i,j)=="false":
                        return "false"
        return "true"
obj=Solution()
obj.isValidSudoku([[ "8", "3", ".", ".", "7", ".", ".", ".", "." ], [ "6", ".", ".", "1", "9", "5", ".", ".", "

```

In []:

```

class Solution:
    def isValidSudoku(self, board):
        def validate(a,i,j):
            l1=[]
            l2=[]
            s1=[0,1,2]
            s2=[3,4,5]
            s3=[6,7,8]
            if i in s1:
                l1=s1
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            elif i in s2:
                l1=s2
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            else:
                l1=s3
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            count=0
            for x in l1:
                for y in l2:
                    if board[x][y].isalnum():
                        if (int(board[x][y])==a):
                            count+=1
            for y in range(9):
                if board[i][y].isalnum():
                    if (int(board[i][y])==a):
                        count+=1
            if (count>2):
                return "false"
        for i in range(9):
            for j in range(9):
                if board[i][j].isalnum():
                    a=int(board[i][j])
                    if validate(a,i,j)=="false":
                        return "false"
        return "true"
obj=Solution()
obj.isValidSudoku([[ "8", "3", ".", ".", "7", ".", ".", ".", "." ], [ "6", ".", ".", "1", "9", "5", ".", ".", "

```

In []:

```

class Solution:
    def groupAnagrams(self, strs):
        l=[]
        for i in strs:
            l+=[''.join(sorted(i))]
        ref=[]
        for i in l:
            if i not in ref:
                ref+=[i]
        ind=[]
        for i in ref:
            inn=[]
            for j in range(len(l)):
                if l[j]==i:
                    inn+=[j]
            ind+=[inn]
        res=[]
        for i in range(len(ind)):
            r=[]
            for j in range(len(ind[i])):
                r+=[strs[ind[i][j]]]
            res+=[r]
        print(res)
org=Solution()
org.groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"])

```

In []:

```

l=['aet', 'aet', 'ant', 'aet', 'ant', 'abt']
print(l.index('aet',0,len(l)-1))

```

In []:

```

a=[1,2,2,1,3,1,2,3]
ref=[]
for i in a:
    if i not in ref:
        ref+=[i]
ind=[]
for i in ref:
    inn=[]
    for j in range(len(a)):
        if a[j]==i:
            inn+=[j]
    ind+=[inn]
ind

```

In []:

```

from itertools import permutations
a="eidbaooo"
b=[''.join(p) for p in permutations('ab')]
c="ba"
a.find(c)

```

In []:

```

from itertools import permutations
class Solution:
    def checkInclusion(self,s1,s2):
        b=[''.join(i) for i in permutations(s1)]
        res=0
        for i in b:
            res=s2.find(i)
            if (res!=-1):
                print(res)
                return 'true'
        if (res==-1):
            print(res)
            return 'false'
obj=Solution()
obj.checkInclusion('ab','eidboaoo')

```

In []:

```

a="satish"
print(a)
b=""
for i in a:
    b=i+b
b

```

In []:

```

import itertools
class Solution:
    def shuffle(self, nums, n):
        l1=nums[:n]
        l2=nums[n:]
        org=[]
        for (i,j) in zip(l1,l2):
            org+= [i,j]
        return org
obj=Solution()
obj.shuffle([1,2,3,4],2)

```

In []:

```

class Solution:
    def countOdds(self, low , high ):
        count=0
        if (low%2==1):
            count+=1
        if(high%2==1):
            count+=1
        val=(high-low)//2
        if (count==2):
            return count+val-1
        else:
            return count+val
obj=Solution()
obj.countOdds(3,7)

```

In []:

```
class Solution:
    def addBinary(self, a,b):
        a=int(a,2)
        b=int(b,2)
        print(a,b)
        return bin(a+b)[2:]
obj=Solution()
obj.addBinary("11","1")
```

In []:

```
class Solution:
    def tribonacci(self, n):
        val=0
        a,b,c=0,1,1
        if(n==0):
            return 0
        elif (n==1 or n==2):
            return 1
        else:
            for i in range(2,n):
                val=a+b+c
                a=b
                b=c
                c=val
            return val
obj=Solution()
obj.tribonacci(25)
```

In []:

```
for i in range(2,3):
    print(i)
```

In []:

```
(75025+46368+121393)/2
```

In []:

```
1389537
```


In []:

```
class Solution:
    def containsDuplicate(self, nums):
        l=[]
        for i in nums:
            if i not in l:
                l+=[i]
        if(l==nums):
            return 'false'
        else:
            return 'true'
obj=Solution()
obj.containsDuplicate([1,2,3,4])
```

In []:

In []:

```
a=[1,2,3]
b=[1,2,3]
a==b
```

In []:

```
class Solution:
    def isAnagram(self, s,t):
        if(sorted(s)==sorted(t)):
            return 'true'
        else:
            return 'false'
obj=Solution()
obj.isAnagram('rat','car')
```

In []:

```
a='rat'
b='tar'
print(sorted(b))
```

In []:

```
a=[200]
print(list(map(int,str(a[0]))))
```

In []:

```
class Solution:
    def isHappy(self, n):
        a=[n]
        b=list(map(int,str(a[0])))
        while(len(b)!=1):
            val=0
            for i in b:
                val+=(i*i)
            a=[val]
            b=list(map(int,str(a[0])))
        if(b[0]==1):
            return 'true'
        else:
            return 'false'

obj=Solution()
obj.isHappy(2)
```

In []:

```
class Solution:
    def isHappy(self, n):
        a=[n]
        b=list(map(int,str(a[0])))
        while(len(b)!=1):
            val=0
            for i in b:
                val+=i
            a=[val]
            b=list(map(int,str(a[0])))
        return b[0]

obj=Solution()
obj.isHappy(0)
```

In []:

```
class Solution:
    def isUgly(self, n):
        l=[2,3,5]
        i=0
        while True:
            while(i<len(l)):
                if(n%l[i]==0):
                    n=n//l[i]
                    i=0
                else:
                    i+=1
            if(n==1):
                return 'true'
            else:
                return 'false'

obj=Solution()
obj.isUgly(1)
```

In []:

```
class Solution:
    def isUgly(self, n: int) -> bool:
        if n == 0: return False
        sieve = (2, 3, 5)
        while n != 1:
            for f in sieve:
                if n % f == 0:
                    n = n // f
                    print(n)
                    break
            else:
                print(n)
                return False
        return True
obj=Solution()
obj.isUgly(14)
```

In []:

```
class Solution:
    def missingNumber(self, nums):
        a=[]
        for i in range(len(nums)+1):
            a+= [i]
        for i in a:
            if i not in nums:
                return i
obj=Solution()
obj.missingNumber([3,0,1])
```

In []:

```
class Solution:
    def moveZeroes(self, nums):
        """
        Do not return anything, modify nums in-place instead.
        """
        count=0
        a=[]
        for i in nums:
            if (i==0):
                count+=1
            else:
                a+= [i]
        for i in range(count):
            a+= [0]
        return a
obj=Solution()
obj.moveZeroes([0,1,0,3,12])
```

In []:

```
import math
class Solution:
    def isPowerOfThree(self, n):
        if(n>0):
            a=(math.log10(n)/math.log10(3))
            if ((a-int(a))==0):
                return 'true'
            else:
                return 'false'
obj=Solution()
obj.isPowerOfThree(243)
```

In []:

```
import math
class Solution:
    def isPowerOfTwo(self, n):
        if(n>0):
            a=(math.log10(n)/math.log10(2))
            if ((a-int(a))==0):
                return True
            else:
                return False
obj=Solution()
obj.isPowerOfTwo(2)
```

In []:

```
import math
class Solution:
    def isPerfectSquare(self, n):
        if(n==0):
            return 'true'
        elif(n>0):
            a=n**(1/2)
            if ((a-int(a))==0):
                return 'true'
            else:
                return 'false'
obj=Solution()
obj.isPerfectSquare(0)
```

In []:

```
a="asd"
print(''.join(sorted(a)))
```

In []:

```

class Solution:
    def canConstruct(self, ransomNote, magazine):
        r=sorted(ransomNote)
        m=sorted(magazine)
        print(r,m)
        for i in range(len(r)):
            if (r[i] in m):
                m.index(r[i])
                if (a== -1):
                    return 'false'
                else:
                    m.remove(m[a])
                    r.remove(r[i])
            if(len(r)==0):
                return 'true'
obj=Solution()
obj.canConstruct('aa','aab')

```

In []:

```

class Solution:
    def toHex(self, num):
        print(hex(num))
obj=Solution()
obj.toHex(26)

```

In []:

```

class Solution:
    def thirdMax(self, nums):
        org=[]
        for i in nums:
            if i not in org:
                org+= [i]
        org.sort()
        if (len(org)<3):
            return (org[len(org)])
        else:
            return (org[-3])
obj=Solution()
obj.thirdMax([3,2,1])

```

In []:

```

class Solution:
    def addStrings(self, num1, num2):

```

In []:

```

ord('1')
type(chr(49))

```

In []:

```

class Solution:
    def addStrings(self, num1, num2):
        m=0
        n=0
        for i in range(len(num1)):
            a=ord(num1[len(num1)-1-i])-48
            m+=(10**i*a)
        for i in range(len(num2)):
            a=ord(num2[len(num2)-1-i])-48
            n+=(10**i*a)
        return (str(m+n))
obj=Solution()
obj.addStrings('456','77')

```

In []:

```

class Solution:
    def countSegments(self, s):
        a=s.split()
        return len(a)
obj=Solution()
obj.countSegments("Hello, my name is John")

```

In []:

```

class Solution:
    def findDisappearedNumbers(self, nums):
        nums.sort()
        a=[]
        for i in nums:
            if i not in a:
                a+=[i]
        mx=max(a)
        mn=min(a)
        r=[]
        if(len(a)==1):
            return a[0]+1
        else:
            for i in range(mn,mx+1):
                if i not in a:
                    r+=[i]
            return r
obj=Solution()
obj.findDisappearedNumbers([1,1])

```

In []:

```

class Solution:
    def findComplement(self,num):
        a=bin(num)[2:]
        x=a.replace('1','2')
        y=x.replace('0','1')
        z=y.replace('2','0')
        return (int(z,2))
obj=Solution()
obj.findComplement(5)

```

In []:

```
class Solution:
    def findMaxConsecutiveOnes(self, nums):
        count=[]
        c=0
        for i in nums:
            if (i==1):
                c+=1
                if (i==nums[len(nums)-1]):
                    count+=c]
            else:
                count+=c]
                c=0
        return (max(count))
obj=Solution()
obj.findMaxConsecutiveOnes([1,1,0,1,1,1])
```

In []:

```
class Solution:
    def findWords(self, words):
        a="qwertyuiop"
        b="asdfghjkl"
        c="zxcvbnm"
        a=sorted(a)
        b=sorted(b)
        c=sorted(c)
        print(a,b,c)
        org=[]
        for i in words:
            i=i.lower()
            word=sorted(i)
            dup=[]
            for j in word:
                if j not in dup:
                    dup+=j]
            org+=dup]
        return org
obj=Solution()
obj.findWords(["Hello", "Alaska", "Dad", "Peace"])
```

In []:

```
while(num!=0):
```

In []:

```
14//2
```

In []:

```

class Solution:
    def checkPerfectNumber(self, num):
        l=[]
        for i in range(1,(num//2)+1):
            if (num%i==0):
                l+=[i]
        if(sum(l)==num):
            return 'true'
        else:
            return 'false'
obj=Solution()
obj.checkPerfectNumber(99999993)

```

In []:

```

class Solution:
    def matrixReshape(self, mat, r, c):
        a=len(mat)
        b=len(mat[0])
        l=[]
        for i in range(a):
            for j in range(b):
                l+=mat[i][j]
        if (a*b==r*c):
            org=[[0 for i in range(c)] for j in range(r)]
            k=0
            for i in range(r):
                for j in range(c):
                    org[i][j]=l[k]
                    k+=1
            return org
        else:
            return mat
obj=Solution()
obj.matrixReshape([[1,2,3],[4,5,6]],2,2)

```

In []:

```

class Solution:
    def maximumProduct(self, nums):
        product=1
        nums.sort()
        if (nums[0]<0 and nums[1]<0 and nums[len(nums)-1]<-1*nums[0] and nums[len(nums)-1]<
            for i in range(2):
                nums[i]=-1*nums[i]
        nums.sort()
        for i in range(3):
            a=max(nums)
            product*=a
            nums.remove(a)
        return product
obj=Solution()
obj.maximumProduct([-1,-2,1,2,3])

```


In []:

```

class Solution:
    def findMaxAverage(self, nums,k):
        nums.sort()
        summ=0
        l=[]
        for i in range(len(nums)-k+1):
            summ=nums[i]+nums[i+1]+nums[i+2]+nums[i+3]
            print(summ)
            l+=[summ/k]
        return(max(l))
obj=Solution()
obj.findMaxAverage([1,12,-5,-6,50,3],4)

```

In []:

```

class Solution:
    def singleNonDuplicate(self, nums):
        for i in nums:
            if(nums.count(i)==1):
                return i
obj=Solution()
obj.singleNonDuplicate(
[1,1,2,2,3,3,4,4,5,5,6,6,7,7,8,8,9,9,10,10,11,11,12,12,13,13,14,14,15,15,16,16,17,17,18,18,

```

In []:

```

a=[1,2,3]
print(''.join(map(str,a))[:-1])

```

In []:

```

a=[1,2,3]
print(''.join(map(str,a))[:-1])
a='123'
print(list(map(int,str([a][0]))))

```

In []:

```

class Solution:
    def flipAndInvertImage(self, image):
        a=len(image)
        l=[]
        for i in range(a):
            dup=image[i]
            p=''.join(map(str,dup))[:-1]
            q=p.replace('0','2')
            r=q.replace('1','0')
            s=r.replace('2','1')
            temp=list(map(int,str([s][0])))
            l+=[temp]
        return l
obj=Solution()
obj.flipAndInvertImage([[1,1,0,0],[1,0,0,1],[0,1,1,1],[1,0,1,0]])

```

In []:

```

class Solution:
    def backspaceCompare(self, s,t):
        si=[]
        ti=[]
        for i in range(len(s)):
            if(s[i]=="#"):
                si+=[]
        for i in range(len(t)):
            if(t[i]=="#"):
                ti+=[]
        s=[i for i in s]
        t=[i for i in t]
        for i in si:
            s[i]="."
            s[i-1]="."
        for i in ti:
            t[i]="."
            t[i-1]="."
        s=''.join(map(str,s))
        t=''.join(map(str,t))
        if(s==t):
            return True
        else:
            return False
obj=Solution()
obj.backspaceCompare("ab#c", "ad#c")

```

In []:

```

import itertools
class Solution:
    def buddyStrings(self, s, goal):

```

In []:

```

class Solution:
    def transpose(self, matrix):
        a=len(matrix)
        l=[]
        for i in range(a):
            for j in range(a):
                l+=[matrix[i][j]]
        org=[]
        for i in range(len(matrix[0])):
            tem=[]
            for j in range(a):
                tem+=[l[i+a*j]]
            org+=[tem]
        return org
obj=Solution()
obj.transpose([[1,2,3],[4,5,6]])

```

In []:

```
class Solution:
    def rangeSumBST(self, root, low, high):
        for i in root:
            print(type(i))
obj=Solution()
obj.rangeSumBST([10,5,15,3,7,null,18],7,15)
```

In []:

```
class Solution:
    def bitwiseComplement(self, n):
        a=bin(n)[2:]
        a=a.replace('0','2')
        a=a.replace('1','0')
        a=a.replace('2','1')
        return int(a,2)
obj=Solution()
obj.bitwiseComplement(5)
```

In []:

```
bin(5)
```

In []:

```
n=234
n=list(map(int,str([n][0])))
a=23
print(list(map(str,str([a][0]))))
```

In []:

```
class Solution:
    def replaceElements(self, arr):
        if(len(arr)==1):
            arr[len(arr)-1]=-1
            return arr
        else:
            for i in range(len(arr)-1):
                arr[i]=max(arr[i+1:])
            arr[len(arr)-1]=-1
            return arr
obj=Solution()
obj.replaceElements([17,18,5,4,3,2])
```

In []:

```
a=[1,2,3]
print(''.join(map(str,a)))
a=412
print(list(map(int,str([a][0]))))
```

In []:

```
num=1234
a=list(map(int,str([num][0])))
a
```

In []:

```

class Solution:
    def arrayRankTransform(self, arr):
        org=[]
        for i in arr:
            if i not in org:
                org+= [i]
        org.sort()
        dic={}
        for i in range(1,len(org)+1):
            dic[org[i-1]]=i
        l=[]
        for i in arr:
            l+= [dic[i]]
        return l
obj=Solution()
obj.arrayRankTransform([40,10,20,30,10,40])

```

In []:

```

class Solution:
    def findOcurrences(self, text, first, second):
        l=text.split(' ')
        org=[]
        for i in range(len(l)-2):
            if (l[i]==first and l[i+1]==second):
                org+= [l[i+2]]
        return org
obj=Solution()
obj.findOcurrences("alice is a good girl she is a good student","a","good")

```

In []:

```

import calendar
class Solution:
    def dayOfYear(self, date):
        d=[31,28,31,30,31,30,31,31,30,31,30,31]
        if(calendar.isleap(int(date[0:4]))):
            d[1]=29
        if(int(date[5:7])==1):
            return int(date[8:10])
        else:
            return (sum(d[0:int(date[5:7])-1])+int(date[8:10]))
obj=Solution()
obj.dayOfYear("2020-01-09")

```

In []:

```

class Solution:
    def countCharacters(self, words, chars):
        org=(sorted(chars))
        l=[]
        for i in words:
            a=sorted(i)
            l+= [a]
        res=0
obj=Solution()
obj.countCharacters(["cat","bt","hat","tree"],"atach")

```

In []:

```
import calendar
class Solution:
    def dayOfTheWeek(self, day, month, year):
        d=[31,28,31,30,31,30,31,31,30,31,30,31]
        days={0:"Sunday", 1:"Monday", 2:"Tuesday", 3:"Wednesday", 4:"Thursday", 5:"Friday",
        if(calendar.isleap(year)):
            d[1]=29
        y=(year-1)%100
        ly=y//7
        nly=y%7
        oy=(ly*2+nly*1)
        om=0
        od=0
        res=0
        if(month==1):
            res=(oy+day)%7
            return (days[res])
        else:
            res=(sum(d[0:month-1])+day+oy)%7
            return (days[res])
obj=Solution()
obj.dayOfTheWeek(18,7,1999)
```

In []:

```
class Solution:
    def uniqueOccurrences(self, arr):
        org=[]
        for i in arr:
            if i not in org:
                org+= [i]
        count=[]
        for i in org:
            c=arr.count(i)
            count+= [c]
        dup=[]
        for i in count:
            if i not in dup:
                dup+= [i]
        if(count==dup):
            return True
        else:
            return False
obj=Solution()
obj.uniqueOccurrences([1,2,2,3,2,1,4])
```

In []:

```

class Solution:
    def arrayRankTransform(self, arr):
        org=[]
        for i in arr:
            if i not in org:
                org+=[i]
        org.sort()
        dic={}
        for i in range(1,len(org)+1):
            dic[org[i-1]]=i
        l=[]
        for i in arr:
            l+=[dic[i]]
        return l
obj=Solution()
obj.arrayRankTransform([1,2,3,2,3])

```

In []:

```

(0 ^ 2 ^ 4 ^ 6 ^ 8)

```

In []:

```

0^285485

```

In []:

```

a=[1,2,3]
b=a
b.remove(1)
print(b)
print(a)

```

In []:

```

class Solution:
    def checkIfExist(self, arr):
        b=arr
        for i in arr:
            print(i)
            b.remove(i)
            print(b)
            for j in b:
                if(i==2*j):
                    return True
        return False
obj=Solution()
obj.checkIfExist([-20,8,-6,-14,0,-19,14,1])

```

In []:

```
class Solution:
    def countNegatives(self, grid):
        a=len(grid)
        c=0
        for i in range(a):
            for j in range(a):
                if(grid[i][j]<0):
                    c+=1
        return c
obj=Solution()
obj.countNegatives([[4,3,2,-1],[3,2,1,-1],[1,1,-1,-2],[-1,-1,-2,-3]])
```

In []:

```
class Solution:
    def canMakeArithmeticProgression(self, arr):
        arr.sort()
        print(arr)
        diff=arr[1]-arr[0]
        print(diff)
        a=0
        for i in range(2,len(arr)):
            a=arr[i]-arr[i-1]
            print(a)
            if(diff!=a):
                return False
        return True
obj=Solution()
obj.canMakeArithmeticProgression([3,5,1])
```

In []:

```
arr=[1,2,3]

b=arr
b.pop()
print(b)
arr
```

In []:

```
arr=[1,4,2,3]
a=sorted(arr)
print(a)
```

In []:

```
class Solution:
    def thousandSeparator(self, n):
        l=[i for i in str(n)]
        l=l[::-1]
        print(l)
        j=0
        i=0
        while(i!=-1):
            if(i==3*(j+1)+j):
                l.insert(i, ".")
                print(l)
                j+=1
                i+=1
            else:
                i+=1
        l=l[::-1]
        return (''.join(l))
obj=Solution()
obj.thousandSeparator(7995998610)
```

In []:

```
a="qwertyui"
list(map(str,a))
```

In []:

```
class Solution:
    def trimMean(self, arr):
        arr.sort()
        a=(len(arr)*5)//100
        del arr[0:a]
        arr=arr[::-1]
        del arr[0:a]
        return (sum(arr)/len(arr))
obj=Solution()
obj.trimMean([6,2,7,5,1,2,0,3,10,2,5,0,5,5,0,8,7,6,8,0])
```


In []:

```
import itertools
class Solution:
    def slowestKey(self, releaseTimes, keysPressed):
        longkey=""
        longtime=0
        long=0
        s=0
        for i,j in zip(releaseTimes,keysPressed):
            s+=i
            t=s-longtime
            longtime=t
            if(t>=long):
                long=t
                longkey=j
        return longkey
obj=Solution()
obj.slowestKey([9,29,49,50], "cbcd")
```

In []:

```
import itertools
class Solution:
    def slowestKey(self, releaseTimes, keysPressed):
        l=[releaseTimes[0]]
        for i in range(1,len(releaseTimes)):
            diff=releaseTimes[i]-releaseTimes[i-1]
            l+=[diff]
        org=[]
        maxi=max(l)
        for i in range(len(l)):
            if (l[i]==maxi):
                org+=[keysPressed[i]]
        return max(org)
obj=Solution()
obj.slowestKey([9,29,49,50], "cbcd")
```

In []:

```
max(['a','b','c'])
```

In []:

```
class Solution:
    def getMaximumGenerated(self, n):
        l=[0,1]
        if (n==0):
            return 0
        if (n==1):
            return 1
        c=2
        while(c!=n+1):
            l+=l[c//2]+l[c%2]
            c+=1
        return max(l)
obj=Solution()
obj.getMaximumGenerated(7)
```

In []:

```
' '.join(sorted("asdfg"))
```

In []:

```
class Solution:
    def countConsistentStrings(self, allowed, words):
        words=' '.join(sorted(words))
        print(words)
        count=0
        for i in words:
            dup=""
            for j in i:
                if j not in allowed:
                    dup+=j
            dup=' '.join(sorted(dup))
            if(words==dup):
                count+=1
        return count
obj=Solution()
obj.countConsistentStrings('ab',["ad","bd","aaab","baa","badab"])
```

In []:

```

class Solution:
    def check(self, nums):
        a=sorted(nums)
        if(a==nums):
            return True
        l=[]
        for i in range(len(nums)):
            m=a[i+1:len(nums)]+a[0:i+1]
            l+=m
        print(l)
        print(nums)
        for i in l:
            if(nums==l):
                return True
        return False
obj=Solution()
obj.check([3,4,5,1,2])

```

In []:

```

class Solution:
    def mergeAlternately(self, word1, word2):
        if(len(word1)>len(word2)):
            diff=abs(len(word1)-len(word2))
            word2=word2+" "*diff
        else:
            diff=abs(len(word1)-len(word2))
            word1=word1+" "*diff
        s=""
        for i in range(len(word1)):
            s=s+word1[i]+word2[i]
        return(s.replace(" ", ""))
obj=Solution()
obj.mergeAlternately("ab", "pqrs")

```

In []:

```

class Solution:
    def findCenter(self, edges):
        l=[]
        for i in edges:
            l+=i
        dup=[]
        for i in l:
            if i not in dup:
                dup+=i
        u=[]
        for i in dup:
            a=l.count(i)
            u+=a
        ind=u.index(max(u))
        return dup[ind]
obj=Solution()
obj.findCenter([[1,2],[5,1],[1,3],[1,4]])

```

In []:

```
max([4,1,1,1,1])
```


In []:

```
class Solution:
    def splitArraySameAverage(self, nums):
        for i in range(len(nums)-1):
            a=nums[0:i+1]
            b=nums[i+1:]
            print(sum(a),sum(b))
            if((sum(a)/len(a))==(sum(b)/len(b))):
                return True
        return False
obj=Solution()
obj.splitArraySameAverage([1,2,3,4,5,6,7,8])
```

In []:

```
class Solution:
    def backspaceCompare(self, s,t):
        si=[]
        ti=[]
        for i in range(len(s)):
            if(s[i]=="#"):
                si+=[]
        for i in range(len(t)):
            if(t[i]=="#"):
                ti+=[]
        s=[i for i in s]
        t=[i for i in t]
        for i in si:
            s[i]="."
            s[i-1]="."
        for i in ti:
            t[i]="."
            t[i-1]="."
        s=''.join(map(str,s))
        t=''.join(map(str,t))
        if(s==t):
            return True
        else:
            return False
obj=Solution()
obj.backspaceCompare("ab#c","ad#c")
```

In []:

```

class Solution:
    def backspaceCompare(self, s,t):
        s=list(map(str,s))
        t=list(map(str,t))
        while(len(s)>1 and "#" in s):
            if (s[0]=='#'):
                s.remove(s[0])
            else:
                ind=s.index("#")
                s.remove(s[ind])
                s.remove(s[ind-1])
        while(len(t)>1 and "#" in t):
            if (t[0]=='#'):
                t.remove(t[0])
            else:
                ind=t.index("#")
                t.remove(t[ind])
                t.remove(t[ind-1])
        s=''.join(s)
        t=''.join(t)
        print(s,t)
        s.replace('#','')
        t.replace('#','')
        print(s,t)
        if(s==t):
            return True
        return False
obj=Solution()
obj.backspaceCompare("oi###mupo##rszty#s#xu###bxx##dq#gdjz","oi###mu#ueo##pk#o##rsztu#y#s#

```

In []:

```

class Solution:
    def checkIfPangram(self, sentence):
        alp="abcdefghijklmnopqrstuvwxyz"
        new=""
        for i in sentence:
            if i not in new:
                new+=i
        new=sorted(new)
        print(new,alp)
        if(alp==new):
            return True
        return False
obj=Solution()
obj.checkIfPangram("thequickbrownfoxjumpsoverthelazydog")

```

In []:

```
import gmpy2
```

In []:

```
class Solution:
    def sortSentence(self, s):
        l=s.split(' ')
        org=[]
        for i in l:
            org+=[i[::-1]]
        org=sorted(org)
        a=len(org)
        new=[]
        for i in org:
            new+=[i[::-1]]
        new=' '.join(new)
        for i in range(1,a+1):
            j=str(i)
            new=new.replace(j,'')
        return new
obj=Solution()
obj.sortSentence("is2 sentence4 This1 a3")
```

In []:

```
class Solution:
    def isSumEqual(self, firstWord, secondWord, targetWord):
        s="abcdefghij"
        fs=""
        ss=""
        ts=""
        for i in firstWord:
            ind=s.index(i)
            fs+=str(ind)
        for i in secondWord:
            ind=s.index(i)
            ss+=str(ind)
        for i in targetWord:
            ind=s.index(i)
            ts+=str(ind)
        res=int(fs)+int(ss)
        if(int(ts)==res):
            return True
        return False
obj=Solution()
obj.isSumEqual('acb','cba','cdb')
```


In []:

```

class Solution:
    def secondHighest(self, s):
        l=[]
        for i in s:
            if (i.isnumeric()):
                l+= [i]
        l=sorted(l,reverse=True)
        org=[]
        for i in l:
            if i not in org:
                org+= [i]
        if(len(org)>1):
            return int(org[1])
        return -1
obj=Solution()
obj.secondHighest("dfa12321afd")

```

In []:

```

class Solution:
    def toHex(self, num):
        print(hex(num))
obj=Solution()
obj.toHex(26)

```

In []:

```

a=int('1111'*8,2)
hex(a)

```

In []:

```

class Solution:
    def maxAscendingSum(self, nums):
        l=[]
        for i in range(len(nums)-1):
            a=nums[i]
            s1=[a]
            flag=1
            for j in range(i+1,len(nums)):
                if(nums[j]>a and flag==1):
                    a=nums[j]
                    s1+= [a]
                else:
                    flag=0
            l+= [s1]
        su=[]
        for i in l:
            su+= [sum(i)]
        return max(su)
obj=Solution()
obj.maxAscendingSum([10,20,30,5,10,50])

```

In []:

```

class Solution:
    def numDifferentIntegers(self, word):
        i=0
        l=[]
        s=""
        while(i<len(word)):
            a=word[i].isdigit()
            if(a):
                flag=1
            elif(a is False):
                flag=0
            if(flag==1):
                s+=word[i]
                i+=1
            elif(flag==0):
                if(s!=''):
                    l+=[s]
                    s=""
                    i+=1
            if(i==len(word)):
                if(s!=''):
                    l+=[s]
        li=[]
        for i in l:
            a=int(i)
            if a not in li:
                li+=[a]
        return len(li)
obj=Solution()
obj.numDifferentIntegers("leet1234code234")

```

In []:

```

class Solution:
    def isCovered(self, ranges, left, right):
        s=[]
        for i in range(left,right+1):
            s+=[i]
        l=[]
        for i in ranges:
            l+=i
        org=[]
        for i in l:
            if i not in org:
                org+=[i]
        org=sorted(org)
        for i in s:
            if (i<org[0] or i>org[len(org)-1]):
                return False
        return True
obj=Solution()
obj.isCovered([[37,49],[5,17],[8,32]],29,49)

```

In []:

```
class Solution:
    def largestOddNumber(self, num):
        l=list(map(int,num))
        org=[]
        for i in l:
            if (i%2!=0):
                org+=[i]
        if (len(org)==0):
            return ""
        return str(max(org))
obj=Solution()
obj.largestOddNumber("52")
```

In []:

```
class Solution:
    def findMiddleIndex(self, nums):
        if(sum(nums[1:])==0):
            return 0
        elif(sum(nums[0:len(nums)-1])==0):
            return len(nums)-1
        for i in range(1,len(nums)):
            a=nums[i-1:i]
            b=nums[i+1:]
            if (sum(a)==sum(b)):
                return i
        return -1
obj=Solution()
obj.findMiddleIndex([2,3,-1,8,4])
```

In []:

```
class Solution:
    def maximumDifference(self, nums):
        mx=[]
        for i in range(len(nums)-1):
            a=nums[i]
            b=nums[i+1:]
            m=max(b)
            if (m>a):
                diff=m-a
                mx+=[diff]
        return max(mx)
obj=Solution()
obj.maximumDifference([7,1,5,4])
```

In []:

```
class Solution:
    def construct2DArray(self, original, m, n):
        if (m==1 and n==1):
            return original
        if (len(original)!=m*n):
            return [[]]
        l=[]
        k=0
        for i in range(m):
            sub=[]
            for j in range(n):
                sub+= [original[k]]
                k+=1
            l+= [sub]
        return l
obj=Solution()
obj.construct2DArray([1,2],1,1)
```

In []:

```
class Solution:
    def areNumbersAscending(self, s):
        l=[]
        flag=1
        s=""
        for i in s:
            if(i.isdigit()):
                flag=1
                if (flag==1):
                    s+=i
            else:
                flag=0
                nu=int(s)
                l+= [nu]
                s=""
        a=sorted(l)
        org=[]
        for i in a:
            if i not in org:
                org+= [i]
        print(a,org)
        if(a==l and org==a):
            return True
        return False
obj=Solution()
obj.areNumbersAscending("hello world 5 x 5")
```

In []:

```
class Solution:
    def divideString(self, s, k, fill):
        l=len(s)
        rem=0
        if(l%k!=0):
            l=(l//k)*k
            rem=len(s)-l
        org=[]
        i=0
        while(i<l):
            word=s[i:i+k]
            org+=word
            i+=k
        if(rem!=0):
            let=(k-rem)*fill
            extra=s[(len(s)-rem):]
            org+=extra+let
        return org
obj=Solution()
obj.divideString('abcdefghij',3,'x')
```

In []:

```
a=[1,2,3,4]
a[1]=5
a
```

In []:

```

class Solution:
    def sortEvenOdd(self, nums):
        oddind=[]
        evenind=[]
        odd=[]
        even=[]
        for i in range(len(nums)):
            if(nums[i]%2==0):
                evenind+= [i]
                even+= [nums[i]]
            else:
                oddind+= [i]
                odd+= [nums[i]]
        odd=sorted(odd,reverse=True)
        even=sorted(even)
        print(odd,even)
        print(oddind,evenind)
        for i in range(len(odd)):
            nums[oddind[i]]=odd[i]
        for i in range(len(even)):
            nums[evenind[i]]=even[i]
        return nums
obj=Solution()
obj.sortEvenOdd([36,45,32,31,15,41,9,46,36,6,15,16,33,26,27,31,44,34])
#[6,45,16,41,33,31,31,26,32,34,27,36,15,36,15,9,44,46]
#[9,46,15,45,15,41,27,34,32,31,33,31,36,26,36,16,44,6]

```

In []:

```

class Solution:
    def countOperations(self, num1, num2):
        if (num1==0 or num2==0):
            return 0
        c=0
        flag=1
        while(flag==1):
            if(num1>num2):
                num1=num1-num2
                c+=1
            else:
                num2=num2-num1
                c+=1
        return c
obj=Solution()
obj.countOperations(2,3)

```

In [3]:

```

a=23
print(list(map(str,str([a][0]))))

['2', '3']

```

In []:

```

y=00010
y

```

In []:

```
a=3+6j
a.real
a.imag
```

In []:

```
print(int(True))
print(int(False))
True+True+False+True+True
```

In []:

```
bool(000010000.90)
```

In []:

```
a=1
b=2
while(a==1 and b!=2):
    print(a,b)
    break
```

In []:

```
d={3:'xsa',9:'fdf',1:'fvefgve'}
sorted(d)
```

In []:

```
class Solution:
    def sortPeople(self, names, heights):
        d={}
        k=0
        for i in heights:
            d[i]=names[k]
            k+=1
        print(d.keys())
        res=[]
obj=Solution()
obj.sortPeople(["Mary", "John", "Emma"],[180,165,170])
```

In []:

```

class Solution:
    def findMaxK(self, nums):
        pl=[]
        nl=[]
        for i in nums:
            if(i<0):
                nl+=[i]
            else:
                pl+=[i]
        pl=sorted(pl,reverse=True)
        for i in pl:
            a=-1*i
            if(a in nl):
                return i
obj=Solution()
obj.findMaxK([-1,2,-3,3])

```

In []:

```

class Solution:
    def isCircularSentence(self, sentence: str) -> bool:
        l=len(sentence)
        if(sentence[0]==sentence[l-1]):
            a=sentence.split(' ')
            if(len(a)==1):
                return True
            for i in range(len(a)-1):
                b=len(a[i])
                if(a[i][b-1]!=a[i+1][0]):
                    return False
            return True
obj=Solution()
obj.isCircularSentence("ab a a")

```

In []:

```

a=[1,2,3,2,3,2]
a.remove(2)
a

```

In []:

```

a="11a1"
a.isdigit()

```


In []:

```
class Solution:
    def maximumValue(self, strs):
        l=[]
        for i in strs:
            if(i.isdigit()):
                a=int(i)
                l+=a
            else:
                a=len(i)
                l+=a
        return max(l)
obj=Solution()
obj.maximumValue(["alic3", "bob", "3", "4", "00000"])
```

In []:

```
a="1123"
print(list(map(int,[a][0])))
```

In []:

```
class Solution:
    def alternateDigitSum(self, n: int) -> int:
        l=list(map(int,str([n][0])))
        s=0
        k=0
        print(l)
        for i in l:
            a=i*(-1**k)
            print(a)
            k+=1
            print(k)
            s+=a
        return s
obj=Solution()
obj.alternateDigitSum(521)
```

In []:

```
(-1)**0
```

In []:

```

class Solution:
    def transpose(self, matrix):
        a=len(matrix)
        b=len(matrix[0])
        l=[]
        for i in range(a):
            for j in range(b):
                l+=matrix[i][j]
        org=[]
        for i in range(len(matrix[0])):
            tem=[]
            for j in range(a):
                ind=i+b*j
                tem+=l[ind]
            org+=tem
        return org
obj=Solution()
obj.transpose([[1,2,3],[4,5,6]])

```

In []:

```

import calendar
class Solution:
    def dayOfTheWeek(self, day, month, year):
        d=[3,0,3,2,3,2,3,3,2,3,2,3]
        days={0:"Sunday", 1:"Monday", 2:"Tuesday", 3:"Wednesday", 4:"Thursday", 5:"Friday",
        if(calendar.isleap(year)):
            d[1]=1
        y=year%100
        print(y)
        leap=(y-1)//4
        print(leap)
        nonleap=(y-leap-1)
        print(nonleap)
        odd=((leap*2)+nonleap+day+sum(d[0:month-1]))
        print(odd%7)
        return days[odd]
obj=Solution()
obj.dayOfTheWeek(18,7,1999)

```

In []:

```

d=[3,0,3,2,3,2,3,3,2,3,2,3]
sum(d[:7])
#[82597, -9243, 62390, 83030, -97960, -26521, -61011, 83390, -38677, 12333, 75987, 46091, 83794, 19355, -

```

In []:

```

import numpy as np
class Solution:
    def threeSum(self, nums):
        nums=np.array(nums)
        l=len(nums)
        nums=np.sort(nums)
        s=[]
        for i in range(l-2):
            for j in range(i+1,l-1):
                for k in range(j+1,l):
                    sub=[]
                    if(nums[i]+nums[j]+nums[k]==0):
                        sub=np.array([nums[i],nums[j],nums[k]])
                        sub=np.sort(sub)
                        sub=sub.tolist()
                        if sub not in s:
                            s+=sub]
        return s
obj=Solution()
obj.threeSum([-1,0,1,2,-1,-4])

```

In []:

```

from itertools import combinations
class Solution:
    def combine(self, n, k):
        nums=[i for i in range(1,n+1)]
        return list(map(list,itertools.combinations(nums,k)))
obj=Solution()
obj.combine(4,2)

```

In []:

```

class Solution:
    def compress(self, chars):
        org=[]
        for i in chars:
            if i not in org:
                org+=i]
        res=[]
        for i in org:
            ind=chars.index(i)
            c=chars.count(i)
            if (c>1):
                cl=list(map(str,str([c][0])))
                res+=i]+cl
            else:
                res+=i]
        return res
obj=Solution()
obj.compress(['a','a','b','b','c','c','c'])

```

In []:

```

class Solution:
    def reverseWords(self, s):
        li=s.split(" ")
        org=[i for i in li if i!='']
        print(org)
        org=org[::-1]
        a=" ".join(org)
        return a
obj=Solution()
obj.reverseWords("a good example")

```

In []:

```

class Solution:
    def rotate(self, nums,k):
        l=len(nums)
        i=0
        while(i<k):
            org=[]
            org.append(nums[l-1])
            for j in range(l-1):
                org.append(nums[j])
            nums=org
            i+=1
        return nums
obj=Solution()
obj.rotate([1,2,3,4,5,6,7],3)

```

In []:

0&5&6&7

In []:

```

class Solution:
    def rangeBitwiseAnd(self, left, right):
        res=left
        for i in range(left+1,right+1):
            res=res&i
        return res
obj=Solution()
obj.rangeBitwiseAnd(5,7)

```

In []:

5&900

In []:

```

class Solution:
    def rangeBitwiseAnd(self, left: int, right: int) -> int:
        c, i, j = 0, left, right
        AND = i&j
        i += 1
        j -= 1
        c += 1
        while i<=j and c < 1000:
            c += 1
            AND &= i&j
            if AND == 0: return 0
            i += 1
            j -= 1
        return 0 if c >= 1000 else AND

```

In []:

```

class Solution:
    def rangeBitwiseAnd(self, left, right):
        shift = 0
        while left < right:
            left >>= 1
            right >>= 1
            shift += 1
        return left << shift
obj=Solution()
obj.rangeBitwiseAnd(5,7)

```

In []:

```

for i in range(2,1):
    print(i)

```

In []:

```

class Solution:
    def countPrimes(self, n: int) -> int:
        seen, ans = [0] * n, 0
        for num in range(2, n):
            if seen[num]:
                continue
            ans += 1
            a=num*num
            b=((n - 1) // num - num + 1)
            print(b)
            seen[a:n:num] = [1] * b
            print(seen)
        return ans
obj=Solution()
obj.countPrimes(10)

```

In []:

```

nums=[1,2,3,4]
org=[i for i in nums]
org.append(5)
nums

```

In []:

```
l=[1,2,3,4,5,6]
for i in range(5,-1,-1):
    print(i,l[i])
```

In []:

```
class Solution:
    def searchMatrix(self, matrix, target):
        row=len(matrix)
        col=len(matrix[0])
        flag=1
        i=0
        c=0
        r=0
        while(i<col and flag==1):
            if(matrix[0][i]>=target):
                c=i
                flag=0
                c=i
                i+=1
            flag=1
            i=0
            while(i<row and flag==1):
                if(matrix[i][0]>=target):
                    r=i
                    flag=0
                    r=i
                    i+=1
            print(r,c)
            for i in range(r+1):
                for j in range(c+1):
                    if(matrix[i][j]==target):
                        return True
            return False
obj=Solution()
obj.searchMatrix([[1,4],[2,5]],5)
```

In []:

```
min(1,2,3)
```

In []:

```
a=[1,2,3]
a[0:1]
```

In []:

```

class Solution:
    def countSubarrays(self, nums, minK, maxK):
        l=len(nums)
        c=0
        for i in range(l-1):
            if(nums[i]==minK and nums[i]==maxK):
                c+=1
            x=[]
            for j in range(i+1,l):
                x=nums[i:j+1]
                if(min(x)==minK and max(x)==maxK):
                    c+=1
            if(nums[-1]==minK and nums[-1]==maxK):
                c+=1
        return c
obj=Solution()
obj.countSubarrays([1,1,1,1],1,1)

```

In []:

```

class Solution:
    def findDuplicate(self, nums):
        org=[0]*(len(nums))
        for i in nums:
            if org[i]:
                return i
            org[i]=1
obj=Solution()
obj.findDuplicate([1,3,4,2,2])

```

In []:

```

a=[0,0,0,0]
if 1:
    print(a)

```

In []:

```

class Solution:
    def findDuplicate(self, nums):
        N = len(nums) - 1
        seen = [0] * (N+1)
        for num in nums:
            if seen[num]:
                return num
            seen[num] = 1
obj=Solution()
obj.findDuplicate([1,3,4,2,2])

```

In []:

```

class Solution:
    def lengthOfLIS(self, nums):
        l=len(nums)
        si=0
        ei=0
        prev=nums[0]
        res=[]
        for i in range(1,l):
            if(nums[i]>prev):
                ei=i
                print("ei-->",ei)
            else:
                print("si-->",si)
                res+= [ei-si+1]
                si=i
                print("si>",si)
            prev=nums[i]
        return max(res)
obj=Solution()
obj.lengthOfLIS([10,9,2,5,3,7,101,18])

```

In []:

```

class Solution:
    def topKFrequent(self, nums,k):
        org=[]
        for i in nums:
            if i not in org:
                org+= [i]
        print(org)
        d={}
        for i in org:
            c=nums.count(i)
            d[i]=c
        print(d)
        return sorted(d)
obj=Solution()
obj.topKFrequent([4,4,7,5,5,5,9,9,9,9],2)

```

In []:

```

class Solution:
    def findKthPositive(self, arr,k):
        i=0
        a=1
        miss=len(arr)+k
        while(i<k):
            if(a in arr):
                a+=1
                i+=1
            miss=a
        return miss
obj=Solution()
obj.findKthPositive([2,3,4,7,11],5)

```


In []:

```
a=['1','2','3','4','10','11','12']
a.sort()
a
```

In []:

```
class Solution:
    def lexicalOrder(self, n):
        l=[]
        for i in range(1,n+1):
            l+=[str(i)]
        l=set(l)
        org=[]
        for i in l:
            if i not in org:
                org+=[i]
        return org
obj=Solution()
obj.lexicalOrder(13)
```

In []:

```
class Solution:
    def lexicalOrder(self, n):
        l=[]
        for i in range(1,n+1):
            l+=[str(i)]
        l=sorted(l)
        return l
obj=Solution()
obj.lexicalOrder(13)
```

In []:

```
class Solution:
    def lastRemaining(self, n):
        l=[i for i in range(1,n+1)]
        i=0
        c=0
        while(len(l)!=1):
            l.remove(l[i])
            i+=1
            c+=1
            if(i>=n-c):
                l=l[::-1]
                i=0
        return l[0]
obj=Solution()
obj.lastRemaining(10)
```

In []:

```

class Solution:
    def isValidSudoku(self, board):
        def validate(a,i,j):
            l1=[]
            l2=[]
            s1=[0,1,2]
            s2=[3,4,5]
            s3=[6,7,8]
            if i in s1:
                l1=s1
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            elif i in s2:
                l1=s2
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            else:
                l1=s3
                if j in s1:
                    l2=s1
                elif j in s2:
                    l2=s2
                else:
                    l2=s3
            count=0
            for x in l1:
                for y in l2:
                    if board[x][y].isalnum():
                        if (int(board[x][y])==a):
                            count+=1
            for y in range(9):
                if board[i][y].isalnum():
                    if (int(board[i][y])==a):
                        count+=1
            for y in range(9):
                if board[y][j].isalnum():
                    if (int(board[y][j])==a):
                        count+=1
            if (count>3):
                return False
        for i in range(9):
            for j in range(9):
                if (board[i][j].isalnum()):
                    a=int(board[i][j])
                    if validate(a,i,j) is False:
                        return False
        return True
obj=Solution()
obj.isValidSudoku([[".", ".", "4", ".", ".", ".", "6", "3", "."],[".", ".", ".", ".", ".", ".", ".", ".", "."],

```

In []:

```
[[".", ".", "4", ".", ".", ".", "6", "3", "."],
 [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
 ["5", ".", ".", ".", ".", ".", ".", ".", "9", "."],
 [".", ".", ".", "5", "6", ".", ".", ".", ".", "."],
 ["4", ".", "3", ".", ".", ".", ".", ".", ".", "1"],
 [".", ".", ".", "7", ".", ".", ".", ".", ".", "."],
 [".", ".", ".", "5", ".", ".", ".", ".", ".", "."],
 [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
 [".", ".", ".", ".", ".", ".", ".", ".", ".", "."],
 [".", ".", ".", ".", ".", ".", ".", ".", ".", "."]]
```

In []:

```
class Solution:
    def firstMissingPositive(self, nums):
        i=1
        nums=set(nums)
        print(nums)
        while (i in nums):
            i+=1
        return i
obj=Solution()
obj.firstMissingPositive([1,2,1,2,0])
```

In []:

```
class Solution:
    def isValid(self, s):
        ob=["(", "[", "{"]
        cb=[")", "]", "}"]
        l=[]
        if s[0] in cb:
            return False
        for i in s:
            if i in ob:
                l+=i
            else:
                ind=cb.index(i)
                if (ob[ind]==l[len(l)-1]):
                    l.pop()
                else:
                    return False
        if (len(l)==0):
            return True
        else:
            return False
obj=Solution()
obj.isValid("(){}{}")
```

In []:

```

class Solution:
    def divide(self, dividend , divisor):
        org1=dividend
        org2=divisor
        num=0
        if (divisor<0 and dividend>0) or (divisor>0 and dividend<0):
            num=1
            divisor=abs(divisor)
            dividend=abs(dividend)
        count=0
        divisor=abs(divisor)
        dividend=abs(dividend)
        if (divisor==1):
            return org1
        while(dividend>=divisor):
            count+=1
            dividend-=divisor
        if num==1:
            return -count
        else:
            return count
obj=Solution()
obj.divide(-2147483648,-2)

```

In []:

```

class Solution:
    def divide(self, dividend , divisor):
        dividend,divisor,count=abs(dividend),abs(divisor),0
        while dividend<=divisor:
            count+=1
            dividend-=divisor

obj=Solution()
obj.divide(-2147483648,-1)

```

In []:

```

a=[9,8,7,3,4,2,1,3,4,5,8,8,8,8]
b=set(a)
b

```

In []:

```

c=sorted(a,key=a.count,reverse=True)
c

```

In []:

```

alph="ABCDEFGHIIJKLMNOPQRSTUVWXYZ"
alph

```

In []:

```

class Solution:
    def moveZeroes(self, n):
        c=n.count(0)
        n.sort()
        for i in range(c):
            n.remove(n[0])
            n.append(0)
        return n
obj=Solution()
obj.moveZeroes([2,1])

```

In []:

```

class Solution:
    def reverseString(self, s):
        l=len(s)
        for i in range(l):
            ele=s[0]
            s.insert(l-i,ele)
            s.remove(ele)
        return s
obj=Solution()
obj.reverseString(["h","e","l","l","o"])

```

In []:

```

class Solution:
    def firstUniqChar(self, s):
        l=len(s)
        for i in range(l):
            if(s.count(s[i])==1):
                return i
obj=Solution()
obj.firstUniqChar("leetcode")

```

In []:

```

class Solution:
    def hammingWeight(self,n):
        n=str(n)
        return (n.count('1'))
obj=Solution()
obj.hammingWeight(1011)

```

In []:

```

class Solution:
    def reverseBits(self, n):
        n=str(n)
        a=n[::-1]
        a=int(a,2)
        return a
obj=Solution()
obj.reverseBits(1111111111111111111111111111101)

```

In []:

```
class Solution:
    def maxProfit(self, prices):
        res=0
        l=len(prices)
        for i in range(l-1):
            a=prices[i+1:]
            m=max(a)
            if(m-prices[i]>res):
                res=m-prices[i]
        return res
obj=Solution()
obj.maxProfit([7,1,5,3,6,4])
```

In []:

```
data=[3,2,5,1]
a=set(data)
print(list(a)[-1])
print(a.pop())
```

In []:

```
class Solution:
    def isIsomorphic(self, s, t):
        dic={}
        l=len(s)
        for i in range(l):
            v=list(dic.keys())
            print(v)
            print(dic)
            if (s[i] in v and dic[s[i]]!=t[i]):
                return False
            dic[s[i]]=t[i]
        return True
obj=Solution()
obj.isIsomorphic("badc","baba")
```

In []:

```

class Solution:
    def containsNearbyDuplicate(self, nums,k):
        l=len(nums)
        s=set(nums)
        for i in s:
            sl=[]
            for j in range(l):
                if nums[j]==i:
                    sl+= [j]
            if(len(sl)>1):
                sl=set(sl)
                a=list(sl)[-1]
                b=sl.pop()
                if(abs(a-b)<=k):
                    return True
        return False
obj=Solution()
obj.containsNearbyDuplicate([-24500,-24499,-24498,-24497,-24496,-24495,-24494,-24493,-24492

```

In []:

```

class Solution:
    def isUgly(self, n):
        n=abs(n)
        if(n==0):
            return False
        if(n==1):
            return True
        l=[2,3,5]
        while(n!=1):
            if(n%2==0):
                n=n//2
            elif(n%3==0):
                n=n//3
            elif(n%5==0):
                n=n//5
            else:
                return False
        return True
obj=Solution()
obj.isUgly(-2147483648)

```

In []:

```

a=bin(5)[2:]
b=a.count('1')
b

```

In []:

```

l=[2,3,2,5,4,2]
l[0]=5
l

```

In []:

```

class Solution:
    def reverseVowels(self, s):
        vow="aeiouAEIOU"
        res=""
        l=len(s)
        c=101
        for i in s:
            if i in vow:
                res+=i
                s=s.replace(i,str(c),1)
                c+=1
        res=res[::-1]
        for i in range(len(res)):
            s=s.replace(str(101+i),res[i],1)
        return s
obj=Solution()
obj.reverseVowels("leetcode")

```

In []:

```

a="asd fvrfeb 102ndfnvjk"
a=a.replace("102","a")
a

```

In []:

```

a=[2,3,4,2,42]
a.sort()
a

```

In []:

```

class Solution:
    def findTheDifference(self, s,t):
        if(s==""):
            return t[0]
        s=''.join(sorted(s))
        t=''.join(sorted(t))
        print(s,t)
        l=len(s)
        for i in range(l):
            print(s[:i+1])
            if(t.find(s[:i+1])== -1):
                return t[i]
        return t[-1]
obj=Solution()
obj.findTheDifference("ae","aea")

```


In []:

```
a=bin(4)[2:]
b=bin(30)[2:]
m=len(a)
n=len(b)
if(m>n):
    b=(m-n)*"0"+b
else:
    a=(n-m)*"0"+a
a,b
```

In []:

```
n=122122
l=[1,n]
for i in range(2,n//2+1):
    if(n%i==0):
        l+=i
print(l)
```

In []:

```
class Solution:
    def constructRectangle(self, area):
        l=[1,area]
        for i in range(2,area//2+1):
            if(area%i==0):
                l+=i
        diff=area-1
        m=len(l)
        print(l)
        org=[]
        for i in range(m-1):
            for j in range(i+1,m):
                if (l[i]*l[j]==area and abs(l[i]-l[j])<=diff):
                    diff=abs(l[i]-l[j])
                    if(l[i]>=l[j]):
                        org=[l[i],l[j]]
                    else:
                        org=[l[j],l[i]]
        return org
obj=Solution()
obj.constructRectangle(4)
```

In []:

```

class Solution:
    def findRelativeRanks(self, score):
        a=["Gold Medal","Silver Medal","Bronze Medal"]
        s=sorted(score,reverse=True)
        l=len(score)
        org=[]
        for i in range(l):
            if (i==0):
                org+=a[0]
            elif(i==1):
                org+=a[1]
            elif(i==2):
                org+=a[2]
            else:
                org+=str(i+1)
        res=[]
        for i in score:
            return org
obj=Solution()
obj.findRelativeRanks([10,3,8,9,4])

```

In []:

```
bin(1)
```

In []:

```

a=[1,2,3,2,4,5,2,3,2,2,2,]
max(a,key=a.count)
print(a.index(2))

```

In []:

```

class Solution:
    def findShortestSubArray(self, nums):
        print(nums)
        m=max(nums,key=nums.count)
        print(m)
        st=nums.index(m)
        print(st)
        rev=nums[::-1]
        end=nums.index(m)
        end=len(nums)-end-1
        return len(nums[st:end+1])
obj=Solution()
obj.findShortestSubArray([1,2,2,3,1])

```

In []:

```

class Solution:
    def largeGroupPositions(self, s):
        l=len(s)
        org=[]
        char=s[0]
        falg=1
        c=0
        ind=0
        for i in range(1,l):
            if(s[i]==char):
                char=s[i]
                c=i
            else:
                if(c-ind>2):
                    org+=[[ind,c-1]]
                    char=s[i]
                    ind=c
                if(i==l-1 and c-ind>2):
                    org+=[[ind,c-1]]
        return org
obj=Solution()
obj.largeGroupPositions("aaa")

```

In []:

```

s="cdcax"
list(s)

```

In []:

```

vector<int> sortByParityII(vector<int>& nums) {
    for(int i=0,j=1;i<nums.size();){
        if(nums[i]%2==0 )i+=2;
        else if(nums[j]%2!=0) j+=2;
        else swap(nums[i],nums[j]);
    }
    return nums;
}

```

In []:

```

class Solution:
    def sortArrayByParityII(self, nums):
        l=len(nums)
        even=[]
        odd=[]
        for i in nums:
            if(i%2==0):
                even+= [i]
            else:
                odd+= [i]
        even.sort()
        odd.sort()
        print(even,odd)
        org=[]
        for i in range(l):
            if((i%2==0 and nums[i]%2==0) or (i%2!=0 and nums[i]%2!=0)):
                for i in range(len(even)):
                    org=org+[even[i],odd[i]]
        return org
obj=Solution()
obj.sortArrayByParityII([4,2,5,7])

```

In []:

```

class Solution:
    def validMountainArray(self, arr):
        if(arr==sorted(arr,reverse=True) or arr==sorted(arr)):
            return False
        l=len(arr)
        ind=0
        flag=1
        for i in range(l-1):
            if(arr[i+1]<arr[i] and flag==1):
                ind=i
                flag=0
        a=arr[0:ind+1]
        b=arr[ind+1:]
        org1=[]
        org2=[]
        for i in a:
            if i not in org1:
                org1+= [i]
        for i in b:
            if i not in org2:
                org2+= [i]
        org2=sorted(org2,reverse=True)
        org1.sort()
        if(a==org1 and b==org2):
            return True
        return False
obj=Solution()
obj.validMountainArray([0,3])

```

In []:

```

from itertools import combinations
class Solution:
    def subsetsWithDup(self, nums):
        s=[]
        for i in range(len(nums)+1):
            x=[list(j) for j in combinations(nums,i)]
            for i in x:
                if i not in s:
                    s+= [i]
        return s
obj=Solution()
obj.subsetsWithDup([1,2,2])

```

In []:

```

from itertools import permutations
class Solution:
    def subsetsWithDup(self, nums):
        s=[]
        for i in range(len(nums)+1):
            x=[list(j) for j in permutations(nums,i)]
            print(x)
            for i in x:
                if i not in s:
                    s+= [i]
        return s
obj=Solution()
obj.subsetsWithDup([1,2,2])    #[[],[1],[1,2],[1,2,2],[2],[2,2]]

```

In []:

```

class Solution:
    def duplicateZeros(self, arr):
        l=len(arr)
        i=0
        while(i<l):
            if(arr[i]==0):
                arr.insert(i+1,0)
                print(arr)
                arr.remove(arr[-1])
                i+=2
            else:
                i+=1
        return arr
obj=Solution()
obj.duplicateZeros([1,0,2,3,0,4,5,0])

```

In []:

```

a=[12,3,1,212,122]
a.remove(a[-1])
a.insert(0,2121)
a

```

In []:

```
class Solution:
    def uncommonFromSentences(self, s1, s2):
        l1=s1.split(' ')
        l2=s2.split(' ')
        s=set(l1)
        t=set(l2)
        l=[]
        for i in s:
            if i not in t:
                l+=[i]
        for i in t:
            if i not in s:
                l+=[i]
        return list(set(l))

obj=Solution()
obj.uncommonFromSentences("apple apple", "banana")
```

In []:

```
a=[1,2,3,41,3212,4,234,2342,423]
a[-4:]
```

In []:

a[:**-4**]

In []:

```
class Solution:
    c=0
    def replaceElements(self, arr):
        a=max(arr)
        maxi=arr.index(a)
        for i in range(len(arr)-1):
            if(i!=maxi):
                arr[i]=a
            else:
                a=max(arr[i+1:])
                maxi=arr.index(a)
                arr[i]=a
        arr[len(arr)-1]=-1
obj=Solution()
obj.replaceElements([77478,13317,78454,8545,46279,1194,34206,62974,32079,82607,50316,73930,
```

In []:

```
#a=[99996,99996,99996,99996,99996,99996,99996,99996,99996,99996,99996,99996,99996,999
```

In []:

```
#b=[77478,13317,78454,8545,46279,1194,34206,62974,32079,82607,50316,73930,47601,2262,95462,
```

In []:

```
a={'3':2,'1':'11','2':'32'}  
sorted(a)
```

In []:

```
class Solution:  
    def kWeakestRows(self, mat,k):  
        dic={}  
        val=0  
        for i in mat:  
            c=i.count(1)  
            dic[val]=c  
            val+=1  
        print(dic)  
        keys=sorted(dic)  
        l=[]  
        for i in range(k):  
            l+= [dic[keys[i]]]  
        return l  
obj=Solution()  
obj.kWeakestRows([[1,1,0,0,0],[1,1,1,1,0],[1,0,0,0,0],[1,1,0,0,0],[1,1,1,1,1]],3)
```

In []:

```
a=[1,2,3,4]  
b=a  
l=b.remove(2)  
l
```

In []:

```
bin(2)
```

In []:

```
a=['dxaced','dce','asdw']  
sorted(a,key=len)
```

In []:

```
class Solution:
    def stringMatching(self, words):
        le=len(words)
        data=sorted(words,key=len)
        org=[]
        for i in range(le):
            ele=data[i]
            data.remove(ele)
            j=0
            l=len(data)
            flag=1
            while(j<l):
                if(data[j].find(ele)!=-1 and flag==1):
                    flag=0
                if(flag==0):
                    j+=1
                    org+=ele
                j+=1
            data.insert(i,ele)
        return org
obj=Solution()
obj.stringMatching(["mass","as","hero","superhero"])
```

In []:

```
class Solution:
    def destCity(self, paths):
        start=[]
        end=[]
        for i in paths:
            start+=i[0]
            end+=i[1]
        print(start)
        print(end)
        print(set(start)-set(end))
        return list(set(end)-set(start))[0]
obj=Solution()
obj.destCity([["London","New York"],["New York","Lima"],["Lima","Sao Paulo"]])
```


In []:

```
class Solution:
    def maxPower(self, s):
        l=len(s)
        maxi=1
        c=1
        prev=s[0]
        for i in range(1,l):
            if (s[i]==prev):
                c+=1
            else:
                prev=s[i]
                if(c>maxi):
                    maxi=c
                c=1
        if(c>maxi):
            maxi=c
        return maxi
obj=Solution()
obj.maxPower("cbbccbb")
```

In []:

```
class Solution:
    def isPrefixOfWord(self, sentence, searchWord):
        words=sentence.split(' ')
        for i in words:
            if i.find(searchWord)==0:
                return words.find(i)
        return -1
obj=Solution()
obj.isPrefixOfWord("i love eating burger","burg")
```

In []:

```
"12".isdigit()
```

In []:

```
list("asdf")
```

In []:

```

class Solution:
    def sumOddLengthSubarrays(self, arr):
        l=len(arr)
        org=0
        def recursive(arr,i):
            s=0
            for j in range(l-i+1):
                a=arr[j:j+i]
                s+=sum(a)
            return s
        for i in range(1,l+1):
            res=0
            if(i%2!=0):
                a=recursive(arr,i)
                org+=a
        return org
obj=Solution()
obj.sumOddLengthSubarrays([1,4,2,5,3])

```

In []:

```

a=" practice  makes  perfect"
l=a.split()
l

```

In []:

```

class Solution:
    def reorderSpaces(self, text):
        c=0
        l=len(text)
        for i in text:
            if (i==' '):
                c+=1
        text=text.split()
        le=len(text)
        sp=c//(le-1)
        rem=c%(le-1)
        org=((sp*' ').join(text))+(rem*' ')
        return org
obj=Solution()
obj.reorderSpaces("  this  is  a  sentence ")

```

In []:

```

l=[1,2,3,2,1,2,2,1]
org=[]
org=[i for i in l if i not in org]
org

```

In []:

```

s="cbzxy"
t=s[::-1]
i=s.find('a')
j=-(t.find('a')+1)
print(i,j)

```

In []:

```

class Solution:
    def maxLengthBetweenEqualCharacters(self, s):
        l=len(s)
        t=s[::-1]
        res=-1
        for i in s:
            i=s.find(i)
            print(i)
            j=-(t.find('a')+1)
            print(j)
            val=abs(j-i+1)
            if (val>res):
                res=val
        return res
obj=Solution()
obj.maxLengthBetweenEqualCharacters("cbzxy")

```

In []:

```

x={'a':3,'b':2,'c':1}
dict(sorted(x.items(), key=lambda item:item[1]))

```

In []:

```

class Solution:
    def frequencySort(self, nums):
        d={}
        s=sorted(set(nums),reverse=True)
        for i in s:
            c=nums.count(i)
            d[i]=c
        d=dict(sorted(d.items(), key=lambda item:item[1]))
        keys=list(d.keys())
        values=list(d.values())
        l=len(keys)
        org=[]
        for i in range(l):
            org+=keys[i]*values[i]
        return org
obj=Solution()
obj.frequencySort([2,3,1,3,2])

```

In []:

```

org=[]
org+=3*3
org

```

In []:

```

a=[1,2,9,8,3,4,5,6]
a[-2:]+a[:-2]

```

In []:

```
class Solution:
    def maxRepeating(self, sequence, word):
        sl=len(sequence)
        wl=len(word)
        if(sl==wl):
            if (sequence==word):
                return 1
            return 0
        c=0
        for i in range(sl-wl):
            a=sequence[i:i+wl]
            print(i,a)
            if(a==word):
                print(i)
                c+=1
        return c
obj=Solution()
obj.maxRepeating("aaabaaaabaaabaaaabaaaabaaaabaaaaba", "aaaba")
```

In [3]:

```
a="sacdef"
c=a
a.replace('a','c')
c
```

Out[3]:

'sacdef'

In [14]:

```

class Solution:
    def reformatNumber(self, number):
        s=number.replace('-', '')
        s=s.replace(' ', '')
        l=len(s)
        org=""
        i=0
        while(i<l):
            a=s[i:i+3]
            i+=3
            if(i>=l):
                org+=a
                return org
            else:
                org+=a+'-'
            if(l-i==4):
                org+=s[-4:-2]+'-'+s[-2:]
                return org
            if(l-i==2):
                org+=s[-2:]
                return org
        return org
obj=Solution()
obj.reformatNumber("123 4-5678")

```

Out[14]:

'123-456-78'

In [3]:

```

class Solution:
    def maximumUnits(self, boxTypes, truckSize):
        l=len(boxTypes)
        c=0
        res=0
        for i in range(l):
            c+=boxTypes[i][0]
            if(c<truckSize):
                res+=(boxTypes[i][0]*boxTypes[i][1])
                print(res)
            else:
                res+=(boxTypes[i][0]*boxTypes[i][1])
                print(res)
                rem=c-truckSize
                res-=(rem*boxTypes[i][1])
                return res
obj=Solution()
obj.maximumUnits([[5,10],[2,5],[4,7],[3,9]],10)

```

50
60
88

Out[3]:

81

In [10]:

```
2^3
```

Out[10]:

1

In [13]:

```
d={str(i):0 for i in range(1,10+1)}
s=list("12223345")
for i in s:
    if i in d:
        d[i]+=1
```

Input In [13]

```
list(d.keys)[list(d.values()).index(max(list(d.values())))]
```

SyntaxError: closing parenthesis ')' does not match opening parenthesis '['

In [3]:

```
s="123wc ed"
list(s)
```

Out[3]:

```
['1', '2', '3', 'w', 'c', ' ', 'e', 'd']
```

In [32]:

```
class Solution:
    def countBalls(self, lowLimit, highLimit):
        d={str(i):0 for i in range(1, 46)}
        for i in range(lowLimit, highLimit+1):
            s=str(sum(list(map(int, str(i)))))
            if s in d:
                d[s]+=1
        val=list(d.values())
        return max(val)
obj=Solution()
obj.countBalls(5,15)
```

Out[32]:

2

In [41]:

```
1^132
```

Out[41]:

133

In [26]:

```
l=1234
print(list(map(int,str(l))))
```

```
[1, 2, 3, 4]
```

In [44]:

```
from itertools import combinations
class Solution:
    l=[5,1,6]
    c=[]
    for i in range(4+1):
        c+= [list(j) for j in combinations(l,i)]
    s=0
    for i in c:
        res=0
        for j in i:
            res=res^j
        s+=res
    print(s)
```

```
28
```

In [51]:

```
class Solution:
    def countGoodSubstrings(self, s):
        l=len(s)
        org=[]
        for i in range(l-2):
            org+= [s[i:i+3]]
        c=0
        for i in org:
            a=list(set(i))
            b=list(i)
            a.sort()
            b.sort()
            if(a==b):
                c+=1
        return c
obj=Solution()
obj.countGoodSubstrings('xyzzaz')
```

Out[51]:

```
1
```

In [48]:

```
i='zxde'
print(list(set(i)))
```

```
['e', 'z', 'd', 'x']
```

In [49]:

```
print(list(i))  
['z', 'x', 'd', 'e']
```

In []:

```
#[1,2,3,4],[4,1,2,3]
```

In [6]:

```
class Solution:  
    def getLucky(self, s, k):  
        alph="0abcdefghijklmnopqrstuvwxyz"  
        c=0  
        for i in s:  
            ind=alph.find(i)  
            ind=sum(list(map(int,str(ind))))  
            c+=ind  
        if(k==1):  
            return c  
        for i in range(k-1):  
            c=sum(list(map(int,str(c))))  
        return c  
obj=Solution()  
obj.getLucky('iiii',1)
```

Out[6]:

36

In [8]:

```
s='abcdef'  
print('ab' in s)  
print('bcd' in s)  
print('ef' in s)  
print('bd' in s)
```

True
True
True
False

In [4]:

```

class Solution:
    def findMiddleIndex(self, nums):
        if(sum(nums[1:])==0):
            return 0
        for i in range(1,len(nums)-1):
            a=nums[:i]
            b=nums[i+1:]
            if (sum(a)==sum(b)):
                return i
        if(sum(nums[0:len(nums)-1])==0):
            return len(nums)-1
        return -1
obj=Solution()
obj.findMiddleIndex([3,2,-1,-4,8])

```

Out[4]:

1

In [46]:

```

class Solution:
    def areNumbersAscending(self, s):
        l=[]
        s+='z'
        st=""
        for i in s:
            if(i.isdigit()):
                st+=i
            else:
                if(st!=""):
                    print(st)
                    l+=[int(st)]
                    print(l)
                    st=""
        a=list(set(l))
        print(a)
        l.sort()
        print(l)
        if(a==l):
            return True
        return False
obj=Solution()
obj.areNumbersAscending("4 5 11 26")

```

4

[4]

5

[4, 5]

11

[4, 5, 11]

26

[4, 5, 11, 26]

[26, 11, 4, 5]

[4, 5, 11, 26]

Out[46]:

False

In [50]:

```
class Solution:
    def maxDistance(self, colors):
        res=0
        l=len(colors)
        for i in range(l-1):
            for j in range(i+1,l):
                print(colors[i],colors[j])
                if (colors[i]!=colors[j] and abs(colors[i]-colors[j])>res):
                    print(colors[i],colors[j])
                    res=abs(colors[i]-colors[j])

        colors=colors[::-1]
        for i in range(l-1):
            for j in range(i+1,l):
                if (colors[i]!=colors[j] and abs(colors[i]-colors[j])>res):
                    res=abs(colors[i]-colors[j])

        return res
obj=Solution()
obj.maxDistance([1,1,1,6,1,1,1])
```

```
1 1
1 1
1 6
1 6
1 1
1 1
1 1
1 1
1 1
1 6
1 1
1 1
1 1
1 1
1 6
1 1
1 1
1 1
1 1
6 1
6 1
6 1
1 1
1 1
1 1
```

Out[50]:

5

In [7]:

```
class Solution:
    def minBitFlips(self, start, goal):
        s=bin(start)[2:]
        g=bin(goal)[2:]
        sl=len(s)
        gl=len(g)
        l=0
        if(sl>gl):
            g="0"*(sl-gl)+g
            l=sl
        else:
            s="0"*(gl-sl)+s
            l=gl
        flips=0
        for i in range(l):
            if(s[i]!=g[i]):
                flips+=1
        return flips
obj=Solution()
obj.minBitFlips(10,7)
```

Out[7]:

3

In [6]:

Input In [6]

111>01111

^

SyntaxError: leading zeros in decimal integer literals are not permitted; use an 0o prefix for octal integers

In [10]:

```

class Solution:
    def convertTime(self, current, correct):
        cur_h=int(current[0:2])
        cur_m=int(current[3:5])
        cor_h=int(correct[0:2])
        cor_m=int(correct[3:5])
        print(cur_h,cur_m,cor_h,cor_m)
        cu_min=cur_h*60+cur_m
        co_min=cor_h*60+cor_m
        diff=co_min-cu_min
        print(diff)
        op=0
        while(diff!=0):
            if(diff>=60):
                op+=1
                diff-=60
            elif(diff>=15):
                op+=1
                diff-=15
            elif(diff>=5):
                op+=1
                diff-=5
            else:
                op+=1
                diff-=1
        return op
obj=Solution()
obj.convertTime("02:30","04:35")

```

2 30 2 30
0

Out[10]:

0

In [3]:

```

m=[2,6,3,1,4,5]
m1=[3,5,8,9,6]
set(m)&set(m1)

```

Out[3]:

{3, 5, 6}

In [6]:

```
s="['ascec','kjn fdkvbner','oiniowaoidw']"
```

In [9]:

```

l=['1','3','2']
s=int(''.join(map(str,l)))
s

```

Out[9]:

132

In [18]:

```

class Solution:
    def removeDigit(self, number, digit):
        ind=[]
        l=len(number)
        for i in range(l):
            if(number[i]==digit):
                ind+= [i]
        org=list(number)
        res=[]
        for i in ind:
            ele=org.pop(i)
            item=int(''.join(map(str,org)))
            res+= [item]
            org.insert(i,ele)
        return str(max(res))
obj=Solution()
obj.removeDigit("133235", '3')

```

Out[18]:

'13325'

In [1]:

```

s='deab'
''.join(sorted(s))

```

Out[1]:

'abde'

In [3]:

```

l=[1,2,3,4]
for i in l:
    print(i)
    l.pop()

```

1
2

In [8]:

```

class Solution:
    def digitCount(self, num):
        l=len(num)
        for i in range(l):
            if (num.count(str(i))!=num[i]):
                return False
        return True
obj=Solution()
obj.digitCount('1210')

```

1 <class 'str'>

Out[8]:

False

In [6]:

```
class Solution:
    def strongPasswordCheckerII(self, password):
        lower=0
        upper=0
        digit=0
        spec=0
        l=len(password)
        for i in range(l):
            if(i<l-1 and password[i]==password[i+1]):
                return False
            if(password[i].islower()):
                lower+=1
            elif(password[i].isupper()):
                upper+=1
            elif(password[i].isdigit()):
                digit+=1
            elif(password[i] in list("!@#$%^&*()-+")):
                spec+=1
        if(password[l-2]==password[l-1]):
            return False
        print(lower,upper,digit,spec)
        if(l>7 and lower>0 and upper>0 and digit>0 and spec>0):
            return True
        return False
obj=Solution()
obj.strongPasswordCheckerII("IloveLe3tcode!")
```

10 2 1 1

Out[6]:

True

In [26]:

```

class Solution:
    def calculateTax(self, brackets, income):
        money=[]
        tax=[]
        for i in brackets:
            money+=i[0]
            tax+=i[1]
        l=len(money)
        val=0
        while(income>0):
            i=0
            s=0
            while(abs(s-money[i])<=income and income!=0):
                income-=money[i]
                print(income)
                val+=abs(s-money[i])*(tax[i]/100)
                s+=money[i]
                print(val)
                i+=1
            else:
                print(income)
                val+=(income)*(tax[i]/100)
                income=0
        return val
obj=Solution()
obj.calculateTax([[3,50],[7,10],[12,25]],10)

```

```

7
1.5
0
1.9
0

```

Out[26]:

```

1.9

```

In [21]:

```

class Solution:
    def decodeMessage(self, key, message):
        org=" "
        alph=" abcdefghijklmnopqrstuvwxyz"
        for i in key:
            if i not in org:
                org+=i
        org=org.replace(" ", '')
        org=" "+org
        res=""
        for i in message:
            ind=org.find(i)
            res+=alph[ind]
        return res
obj=Solution()
obj.decodeMessage("the quick brown fox jumps over the lazy dog","vkbs bs t suepuv")

```

Out[21]:

```

'this is a secret'

```

In [31]:

```

d={'a':0,'b':1}
s="abba"
for i in s:
    d[i]=d[i]+1
    l=list(d.values())
    if 2 in l:
        ind=l.index(2)
        v=list(d.keys())
        print(v[ind])

```

b
a

In [4]:

```

class Solution:
    def findSubarrays(self, nums):
        l=len(nums)
        org=[]
        for i in range(l-1):
            a=nums[i:i+2]
            print(a)
            org+=sum(a)
        s=set(org)
        print(org,s)
        if(len(s)!=len(org)):
            return True
        return False
obj=Solution()
obj.findSubarrays([4,2,4])

```

[4, 2]
[2, 4]
[6, 6] {6}

Out[4]:

True

In [3]:

```

a="satihs"
b=a[::-1]
print(a.find('s'))
print(len(a)-1-b.find('s'))

```

0
5

In [10]:

```

class Solution:
    def checkDistances(self, s, distance):
        k=len(s)
        sind=[]
        lind=[]
        t=s[::-1]
        st=list(set(s))
        for i in st:
            sind+=s.find(i)
            lind+=k-1-t.find(i)
        alph="abcdefghijklmnopqrstuvwxyz"
        l=len(sind)
        for i in range(l):
            if(abs(lind[i]-sind[i]-1)!=distance[alph.find(st[i])]):
                return False
        return True
obj=Solution()
obj.checkDistances("abacccb",[1,3,0,5,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0])

```

Out[10]:

True

In [13]:

```

class Solution:
    def mostFrequentEven(self, nums):
        dic={}
        s=set(nums)
        for i in s:
            if i%2==0:
                dic[i]=nums.count(i)
        keys=list(dic.keys())
        val=list(dic.values())
        print(val)
        m=max(val)
        ind=[]
        for i in range(len(val)):
            if(val[i]==m):
                ind+=keys[i]
        return min(ind)
obj=Solution()
obj.mostFrequentEven([0,1,2,2,4,4,1])

```

[1, 2, 2]

Out[13]:

2

In [15]:

```

s={1,4}
print(s.pop()-s.pop())

```

-3

In [18]:

```

event1 = ["01:15", "02:00"]
event2 = ["02:00", "03:00"]
e1start=int(event1[0][0:2]*60)+int(event1[0][3:])
e1end=int(event1[1][0:2]*60)+int(event1[1][3:])
e2start=int(event2[0][0:2]*60)+int(event2[0][3:])
e2end=int(event2[1][0:2]*60)+int(event2[1][3:])

```

In [19]:

```
a=[[3, -1],[3, -1],[1,1]]
```

TypeErrorTraceback (most recent call last)

Input In [19], in <cell line: 2>()

```

1 a=[[3, -1],[3, -1],[1,1]]
----> 2 set(a)

```

TypeError: unhashable type: 'list'

In [17]:

```

class Solution:
    def oddString(self, words):
        alph="abcdefghijklmnopqrstuvwxyz"
        l=len(words)
        k=len(words[0])
        org={}
        for i in range(l):
            a=words[i]
            res=[]
            for j in range(k-1):
                diff=alph.find(a[j+1])-alph.find(a[j])
                res+=diff
            org[a]=res
        keys=list(org.keys())
        val=list(org.values())
        print(val)
        l=len(val)
        for i in range(l):
            if(val.count(val[i])==1):
                return keys[i]
obj=Solution()
obj.oddString(["dtzca", "dtzca", "dtzca", "yqyyo", "dtzca", "dtzca"])

```

```
[[16, 6, -23, -2], [-8, 8, 0, -10]]
```

Out[17]:

```
'dtzca'
```

In [5]:

```
d={2:90,3:23,1:33}
r=list(d.keys())
l=[[i,d[i]] for i in r]
l
```

Out[5]:

```
[[2, 90], [3, 23], [1, 33]]
```

In [20]:

```
class Solution:
    def mergeArrays(self, nums1,nums2):
        nums=nums1+nums2
        l=len(nums)
        d={}
        keys=[]
        for i in nums:
            keys+=i[0]
        keys.sort()
        for i in keys:
            d[i]=0
        for i in nums:
            if(i[0] in d):
                d[i[0]]+=i[1]
        keys=list(d.keys())
        org=[[i,d[i]] for i in keys]
        return org
obj=Solution()
obj.mergeArrays([[1,2],[2,3],[4,5]],[[1,4],[3,2],[4,1]])
```

Out[20]:

```
[[1, 6], [2, 3], [3, 2], [4, 6]]
```

In [1]:

```
sum([])
```

Out[1]:

```
0
```

In [3]:

```

class Solution:
    def leftRigthDifference(self, nums):
        l=len(nums)
        leftSum=[0]
        rightSum=[0]
        a=nums[0]
        for i in range(1,l):
            leftSum+=a
            a+=nums[i]
        nums=nums[::-1]
        b=nums[0]
        for i in range(1,l):
            rightSum+=b
            b+=nums[i]
        rightSum=rightSum[::-1]
        print(leftSum,rightSum)
obj=Solution()
obj.leftRigthDifference([10])

```

[0] [0]

In [6]:

```

class Solution:
    def splitNum(self, num):
        s=str(num)
        l=len(s)
        org=sorted(s)
        m=""
        n=""
        for i in range(l):
            if i%2==0:
                m+=org[i]
            else:
                n+=org[i]
        print(m,n)
        return int(m)+int(n)
obj=Solution()
obj.splitNum(4325)

```

24 35

Out[6]:

59

In [1]:

```

l=[1]*3+[0]*4+[-1]*5
l

```

Out[1]:

[1, 1, 1, 0, 0, 0, 0, -1, -1, -1, -1, -1]

In [2]:

```
for i in range(1,1):  
    print(i)
```

In [26]:

```
class Solution:  
    def countAndSay(self, n):  
        res='1'  
        for i in range(1,n):  
            res=res+' '  
            l=len(res)  
            prev=res[0]  
            c=1  
            org=""  
            for j in range(1,l):  
                if(res[j]==prev):  
                    c+=1  
                else:  
                    org+=(str(c)+prev)  
                    prev=res[j]  
                    c=1  
            res=org  
        return res  
obj=Solution()  
obj.countAndSay(4)
```

Out[26]:

'1211'

In [28]:

```
class Solution:  
    def merge(self, nums1, m, nums2, n):  
        nums1.extend(nums2)  
        nums1.sort()  
        c=nums1.count(0)  
        nums1=nums1[c:]  
        return nums1  
obj=Solution()  
obj.merge([1,2,3,0,0,0],3,[2,5,6],3)
```

Out[28]:

[1, 2, 2, 3, 5, 6]

In [3]:

```
s1={1,2,3,2,4}  
s2={1,2,}  
s1.intersection(s2)
```

Out[3]:

{1, 2}

In [13]:

```
class Solution:
    def containsNearbyDuplicate(self, nums, k):
        l=len(nums)
        nums1=nums[::-1]
        dup=[]
        for i in nums:
            m=nums.index(i)
            n=l-1-(nums1.index(i))
            temp=[m,n]
            dup+=temp
        print(dup)
        for i in dup:
            print(abs(i[0]-i[1]))
            if(i[0]!=i[1] and abs(i[0]-i[1])<=k):
                return True
        return False
obj=Solution()
obj.containsNearbyDuplicate([1,0,1,1],1)
```

```
[[0, 3], [1, 1], [0, 3], [0, 3]]
3
0
3
3
```

Out[13]:

False

In [16]:

```
s="satish"
s=s.replace('s','',2)
s
```

Out[16]:

'atih'

In [4]:

```

class Solution:
    def largestRectangleArea(self, heights):
        area=[]
        l=len(heights)
        if(l==1):
            return heights[0]
        for i in range(l):
            for j in range(i,l):
                sub=heights[i:j+1]
                print(sub,end=" ")
                m=min(sub)
                print(m,end=" ")
                area+=[m*abs(j-i+1)]
            return max(area)
obj=Solution()
obj.largestRectangleArea([2,1,5,6,2,3])

```

```

[2] 2 [2, 1] 1 [2, 1, 5] 1 [2, 1, 5, 6] 1 [2, 1, 5, 6, 2] 1 [2, 1, 5, 6, 2,
3] 1 [1] 1 [1, 5] 1 [1, 5, 6] 1 [1, 5, 6, 2] 1 [1, 5, 6, 2, 3] 1 [5] 5 [5,
6] 5 [5, 6, 2] 2 [5, 6, 2, 3] 2 [6] 6 [6, 2] 2 [6, 2, 3] 2 [2] 2 [2, 3] 2
[3] 3

```

Out[4]:

10

In [25]:

```

from itertools import combinations
class Solution:
    def subsetsWithDup(self, nums):
        s=set()
        for i in range(len(nums)+1):
            x=[list(j) for j in combinations(nums,i)]
            for i in x:
                s.add(tuple(i))
        #return set(s)
obj=Solution()
obj.subsetsWithDup([1,4,4,4,2,1])

```

In [26]:

```

l=[[-10]]
l[0][0]

```

Out[26]:

-10

In [39]:

```
class Solution:
    def rotate(self, nums,k):
        l=len(nums)
        k=k%l
        nums.extend(nums[:l-k])
        nums.reverse()
        for i in range(l-k):
            nums.pop()
        nums.reverse()
obj=Solution()
obj.rotate([1,2,3,4,5,6,7],3)
```

In [4]:

```
x=[1,2,3,1,2,3]
type([i for i,n in enumerate(x) if n==2][1])
```

Out[4]:

int

In [16]:

```
class Solution:
    def containsNearbyDuplicate(self, nums, k):
        l=len(nums)
        dic={}
        s=set(nums)
        for i in s:
            dic[i]=[]
        for i in range(l):
            dic[nums[i]]+=i
        val=list(dic.values())
        print(val)
        for i in val:
            if len(i)>1:
                if (abs(i[0]-i[1])<=k):
                    return True
        return False
obj=Solution()
obj.containsNearbyDuplicate([1,0,1,1],1)
```

[[1], [0, 2, 3]]

Out[16]:

False

In [18]:

```
s=[1,2,3]
org=[i*5 for i in s]
org
```

Out[18]:

[5, 10, 15]

In [6]:

```

class Solution:
    def maxSubArray(self, nums):
        l=len(nums)
        res=max(nums)
        for i in range(l-1):
            for j in range(i,l):
                sub=sum(nums[i:j+1])
                if(sub>res):
                    res=sub
        return res
obj=Solution()
obj.maxSubArray([1,2,-1,-2,2,1,-2,1])

```

Out[6]:

3

In [11]:

```

class Solution:
    def thousandSeparator(self, n):
        l=[]
        while(n>0):
            rem=n%1000
            l=[rem]+l
            n=n//1000
        le=len(l)
        s=""
        for i in range(le-1):
            s=s+str(l[i])+'.'
        s+=str(l[-1])
        return s
obj=Solution()
obj.thousandSeparator(79959986)

```

Out[11]:

'79.959.986'

In [12]:

```

r="asdfg"
r[-3:]

```

Out[12]:

'dfg'

In [15]:

```

l=['123','003']
l[-1]=str(int(l[-1]))
l

```

Out[15]:

['123', '3']

In [19]:

```

class Solution:
    def thousandSeparator(self, n):
        l=[]
        if(n==0):
            return "0"
        while(n>0):
            rem=n%1000
            rem='000'+str(rem)
            l=[rem[-3:]]+l
            n=n//1000
        l[0]=str(int(l[0]))
        le=len(l)
        s=""
        for i in range(le-1):
            s=s+l[i]+'.'
        s+=l[-1]
        return s
obj=Solution()
obj.thousandSeparator(79959986)

```

Out[19]:

'79.959.986'

In []:

```

from itertools import permutations
class Solution:
    def findEvenNumbers(self, digits):
        a=permutations(digits,3)
        org=[]
        for i in a:
            val=int(''.join(map(str,i)))
            if (val>99 and val%2==0 and val not in org):
                org+=val
        org.sort()
        return org
obj=Solution()
obj.findEvenNumbers([2,1,3,0])

```

In [1]:

```

class Solution:
    def hIndex(self, citations):
        l=len(citations)
        res=[]
        for i in range(1,l+1):
            li=[j for j in citations if j>=i]
            if(i==len(li)):
                res+=i
        return max(res)
obj=Solution()
obj.hIndex([3,0,6,1,5])

```

Out[1]:

3

In [10]:

```
class Solution:
    def getHint(self, secret, guess):
        s1=""
        g1=""
        l=len(secret)
        bulls=0
        cows=0
        for i in range(l):
            if(secret[i]==guess[i]):
                bulls+=1
            else:
                s1+=secret[i]
                g1+=guess[i]
        for i in s1:
            if i in g1:
                cows+=1
                s1=s1.replace(i,'',1)
                g1=g1.replace(i,'',1)
        return str(bulls)+'A'+str(cows)+'B'
obj=Solution()
obj.getHint("1807","7810")
```

Out[10]:

'1A3B'

In [8]:

```
a=[1,2,3,4,5]
a=a[::-1]
res=int(''.join(map(str,a)))
res
```

Out[8]:

54321

In []:

```
n=234
n=list(map(int,str([n][0])))

a=23
print(list(map(str,str([a][0]))))
```

In [12]:

```

class Solution:
    def superPow(self, a, b):
        b=b[::-1]
        print(b)
        res=''.join(map(str,b))
        print(res)
        return (a**res)%1337

```

```

obj=Solution()
obj.superPow(2,[1,0])

```

[0, 1]

01

TypeErrorTraceback (most recent call last)

```

Input In [12], in <cell line: 9>()
      7         return (a**res)%1337
      8 obj=Solution()
----> 9 obj.superPow(2,[1,0])

```

```

Input In [12], in Solution.superPow(self, a, b)
      5 res=''.join(map(str,b))
      6 print(res)
----> 7 return (a**res)%1337

```

TypeError: unsupported operand type(s) for ** or pow(): 'int' and 'str'

In [1]:

```

class Solution:
    def findAnagrams(self,s,p):
        l1=len(p)
        p=set(p)
        l2=len(s)
        print(p)
        org=[]
        print(l2,l1)
        for i in range(l2-l1+1):
            a=set(s[i:i+l1])
            print(a,i)
            if(a==p):
                org+= [i]
        print(org)
obj=Solution()
#obj.findAnagrams("abab","ab")
#obj.findAnagrams("baa","aa")
obj.findAnagrams("ababababab","aab")

```

```

{'a', 'b'}
10 3
{'a', 'b'} 0
{'b', 'a'} 1
{'a', 'b'} 2
{'b', 'a'} 3
{'a', 'b'} 4
{'b', 'a'} 5
{'a', 'b'} 6
{'b', 'a'} 7
[0, 1, 2, 3, 4, 5, 6, 7]

```

In [6]:

```

class Solution:
    def findDuplicates(self, nums):
        dic={}
        for i in nums:
            dic[i]=0
        for i in nums:
            dic[i]+=1
        keys=list(dic.keys())
        vals=list(dic.values())
        l=len(vals)
        org=[]
        for i in range(l):
            if(vals[i]==2):
                org+= [keys[i]]
        return org
obj=Solution()
obj.findDuplicates([4,3,2,7,8,2,3,1])

```

Out[6]:

[3, 2]

In [15]:

```
class Solution:
    def frequencySort(self, s):
        d={}
        a=set(s)
        for i in a:
            d[i]=0
        for i in s:
            d[i]+=1
        org=""
        d=dict(sorted(d.items(), key=lambda item: item[1]))
        for i in d:
            org=i*d[i]+org
        return org
obj=Solution()
obj.frequencySort("tree")
```

Out[15]:

'eetr'

In [19]:

```
s=float('-inf')
type(s)
```

Out[19]:

float

In [1]:

```
bin(2)
```

Out[1]:

'0b10'

In [6]:

```
class Solution:
    def totalHammingDistance(self, nums):
        l=len(nums)
        org=[]
        for i in nums:
            b=bin(i)[2:]
            n=(32-len(b))*'0'+b
            org+=n
        c=0
        for i in range(l-1):
            for j in range(i+1,l):
                a=org[i]
                b=org[j]
                for k in range(32):
                    if(a[k]!=b[k]):
                        c+=1
        return c
obj=Solution()
obj.totalHammingDistance([4,14,2])
```

Out[6]:

6

In [9]:

```

class Solution:
    def nextGreaterElements(self, nums):
        l=len(nums)
        temp=set(nums)
        org=[]
        for i in nums:
            org+=i
        mx=max(org)
        res=[]
        print(org)
        for i in nums:
            ind=org.index(i)
            if i==mx:
                res+=[-1]
            else:
                print(ind)
                res+=org[ind+1]
        return res
obj=Solution()
obj.nextGreaterElements([5,4,3,2,1])

```

[5, 4, 3, 2, 1]

1
2
3
4

IndexErrorTraceback (most recent call last)

Input In [9], in <cell line: 20>()

```

18         return res
19 obj=Solution()
--> 20 obj.nextGreaterElements([5,4,3,2,1])

```

Input In [9], in Solution.nextGreaterElements(self, nums)

```

15         else:
16             print(ind)
--> 17         res+=org[ind+1]
18     return res

```

IndexError: list index out of range

In [8]:

```

class Solution:
    def findMaxLength(self, nums):
        org=[]
        for i in nums:
            org+=bin(i)[2:]
        dup=[]
        for i in org:
            a=i.count('0')
            b=i.count('1')
            print(a,b)
            dup+=[[a,b]]
        c=0
        print(dup)
        l=len(dup)
        for i in range(l):
            start=i+1
            cou=0
            while(start<l):
                if(dup[start]==dup[i]):
                    cou+=1
                else:
                    start=1
                    cou=0
            if(cou>c):
                c=cou
        return c
obj=Solution()
obj.findMaxLength([0,1,2,3,4,5])

```

```

1 0
0 1
1 1
0 2
2 1
1 2
[[1, 0], [0, 1], [1, 1], [0, 2], [2, 1], [1, 2]]

```

Out[8]:

0

In [13]:

```

class Solution:
    def complexNumberMultiply(self, num1, num2):
        x=num1.split('+')
        x[1]=x[1][:-1]
        y=num2.split('+')
        y[1]=y[1][:-1]
        real=(int(x[0])*int(y[0])-int(x[1])*int(y[1]))
        img=(int(x[0])*int(y[1])+int(x[1])*int(y[0]))
        return str(real)+"+"+str(img)+"i"
obj=Solution()
obj.complexNumberMultiply("1+-1i","1+-1i")

```

Out[13]:

'0+-2i'

In [39]:

```

class Solution:
    def nextGreaterElement(self, n):
        if(n<12):
            return -1
        s=str(n)[::-1]
        org="-1"
        l=len(s)
        i=0
        while(i<l-1):
            if(int(s[i])>int(s[i+1])):
                org=(''.join(sorted(s[:i]+s[i+1:],reverse=True))+s[i]+s[i+2:])
                i=1
            else:
                i+=1
        if(org=='-1'):
            return -1
        return int(org[::-1])
obj=Solution()
obj.nextGreaterElement(230241)

```

Out[39]:

230412

In [2]:

```

x=0
y=0
a="x+2-5+y"

```

Out[2]:

'x+2-5+y'

In [3]:

```

num=["satya","sathi","art","air","smn","h"]
print(num)
l=len(num)
for i in range(l-1):
    for j in range(l-i-1):
        if (num[j]>num[j+1]):
            num[j],num[j+1]=num[j+1],num[j]
print(num)

```

```

['satya', 'sathi', 'art', 'air', 'smn', 'h']
['air', 'art', 'h', 'sathi', 'satya', 'smn']

```

In [26]:

```
import array as arr
a=arr.array([1,2,3,4])
b=[1,2,3,4,'i']
print(a,type(a))
print(b,type(b))
```

TypeErrorTraceback (most recent call last)

Input In [26], in <cell line: 2>()

```
1 import array as arr
----> 2 a=arr.array([1,2,3,4])
      3 b=[1,2,3,4,'i']
      4 print(a,type(a))
```

TypeError: array() argument 1 must be a unicode character, not list

In [12]:

```
for i in a:
    print(i)
```

```
1
2
3
4
```

In [2]:

```
class Solution:
    def maximumSwap(self, num):
        mx=num
        s=str(num)
        l=len(s)
        s=list(s)
        for i in range(l-1):
            for j in range(i+1,l):
                if (s[j]>s[i]):
                    s[i],s[j]=s[j],s[i]
                    a=int(''.join(s))
                    if(a>mx):
                        mx=a
                        s[i],s[j]=s[j],s[i]
        return mx
obj=Solution()
obj.maximumSwap(98368)
```

Out[2]:

98863

In [4]:

```

class Solution:
    def repeatedStringMatch(self, a, b):
        sa=set(a)
        sb=set(b)
        s=""
        c=0
        if(sa!=sb):
            return -1
        while(True):
            s+=a
            print(s)
            if(s.find(b)==-1):
                c+=1
            else:
                return c
obj=Solution()
obj.repeatedStringMatch("abcd","cdabcdab")

```

```

abcd
abcdabcd
abcdabcdabcd

```

Out[4]:

2

In [1]:

```

a,b,c=[input() for i in range(3)]
print("name:",a)
print("marks1:",int(b))
print("marks2:",int(c))

```

```

sr
45
54
name: sr
marks1: 45
marks2: 54

```

In []:

```

a,b,c=[input() for i in range(1)].split('')
print("name:",a)
print("marks1:",int(b))
print("marks2:",int(c))

```

In [6]:

```

a=input("enter:")
print("name:",a[:])
print("marks1:",int(a[-2]))
print("marks2:",int(a[-1]))

```

```

enter:Mounika31
name: Mounika
marks1: 3
marks2: 1

```

In [15]:

```
x={'a':3,'b':2,'c':5}
a=x.items()
(sorted(a, key=lambda y: y[1],reverse=True))
```

Out[15]:

```
[('c', 5), ('a', 3), ('b', 2)]
```

In [9]:

```
class Solution:
    def canPartitionKSubsets(self, nums, k):
        l=len(nums)
        s=sum(nums)
        print(s)
        res=0
        if(s%k!=0):
            return False
        res=s//k
        print(res)
        for i in range(l-1):
            ele=res-nums[i]
            print(nums[i],ele)
            if ele not in nums[i+1:]:
                return False
        return True
obj=Solution()
obj.canPartitionKSubsets([4,3,2,3,5,2,1],4)
```

```
20
5
4 1
3 2
2 3
3 2
5 0
```

Out[9]:

False

In [12]:

```
print(%.2f,(100))
```

```
Input In [12]
print(%.2f,(100))
      ^
```

SyntaxError: invalid syntax

In [14]:

```

class Solution:
    def numSubarrayProductLessThanK(self, nums, k):
        c=0
        l=len(nums)
        def product(a):
            print(a)
            val=1
            for x in a:
                val*=x
            return val
        for i in range(l):
            for j in range(i,l):
                if product(nums[i:j+1])<k:
                    c+=1
        return c
obj=Solution()
obj.numSubarrayProductLessThanK([5,2,4,3],5)

```

```

[5]
[5, 2]
[5, 2, 4]
[5, 2, 4, 3]
[2]
[2, 4]
[2, 4, 3]
[4]
[4, 3]
[3]

```

Out[14]:

3

In [5]:

```

class Solution:
    def findLength(self, nums1, nums2):
        s=''.join(map(str,nums1))
        t=''.join(map(str,nums2))
        print(s,t)
        l1=len(s)
        l2=len(t)
        if(l1>l2):
            l=l2
        else:
            l=l1
        mx=0
        for i in range(l1-1,-1,-1):
            for j in range(i+1):

```

```

obj=Solution()
obj.findLength([1,2,3,2,1],[3,2,1,4,7])

```

12321 32147

In [1]:

```

class Solution:
    def operations(self):
        queue=[]
        print("To add enter : 1")
        print("To delete enter : 2")
        print("To display queue enter : 3")
        print("To end Operation enter : -1")
        a=int(input("Select an opearation : "))
        while(True):
            if(a==1):
                ele=int(input("Enter Element to Add : "))
                queue.insert(0,ele)
                print(queue)
                print("element added")
                a=int(input("Select an opearation : "))
            elif(a==2):
                queue.remove(queue[0])
                print(queue)
                print("element deleted")
                a=int(input("Select an opearation : "))
            elif(a==3):
                print(queue)
                print("queue displayed")
                a=int(input("Select an opearation : "))
            elif(a== -1):
                return 'end'
            else:
                print("invalid opearation...please re-enter")
                a=int(input("Select an opearation : "))
obj=Solution()
obj.operations()

```

To add enter : 1
 To delete enter : 2
 To display queue enter : 3
 To end Operation enter : -1
 Select an opearation : m

```

-----
ValueErrorTraceback (most recent call last)
Input In [1], in <cell line: 31>()
    29         a=int(input("Select an opearation : "))
    30 obj=Solution()
----> 31 obj.operations()

Input In [1], in Solution.operations(self)
     6 print("To display queue enter : 3")
     7 print("To end Operation enter : -1")
----> 8 a=int(input("Select an opearation : "))
     9 while(True):
    10     if(a==1):

```

ValueError: invalid literal for int() with base 10: 'm'

In [16]:

```
class Solution:
    def findLeastNumOfUniqueInts(self, arr, k):
        dic={}
        s=set(arr)
        temp=list(s)
        l=sorted(temp)
        for i in s:
            dic[i]=0
        for i in arr:
            dic[i]+=1
        d=dict(sorted(dic.items(),key=lambda dic:dic[1]))
        l=[]
        for i in d:
            l+= [i]*d[i]
        for i in range(k):
            l.remove(l[0])
        return len(set(l))
obj=Solution()
obj.findLeastNumOfUniqueInts([4,3,1,1,3,3,2],3)
```

Out[16]:

2

In [18]:

```
class Solution:
    def kthFactor(self, n, k):
        factors=[]
        for i in range(1,n+1):
            if(n%i==0):
                factors+= [i]
        print(factors)
        if(len(factors)>k):
            return factors[k-1]
        return -1
obj=Solution()
obj.kthFactor(7,2)
```

[1, 7]

Out[18]:

-1

In [7]:

```

class Solution:
    def rangeSum(self, nums, n, left, right):
        l=[]
        for i in range(n):
            s=0
            for j in range(i,n):
                s+=nums[j]
                l+=[s]
        print(l)
        l.sort()
        print(l)
        return sum(l[left-1:right])
obj=Solution()
obj.rangeSum([1,1,1,1],4,1,5)

```

```

[1, 2, 3, 4, 1, 2, 3, 1, 2, 1]
[1, 1, 1, 1, 2, 2, 2, 3, 3, 4]

```

Out[7]:

6

In [12]:

```

class Solution(object):
    def monotoneIncreasingDigits(self, n):
        # First, handle the single-digit cases.
        if n < 10:
            return n

        # Convert the number into a list of integers
        l = []
        for _,c in enumerate(str(n)):
            l.append(int(c))
        n = len(l)
        print(l)
        # Tranverse from right to left
        for i in range(n-1,0,-1):
            if l[i] < l[i-1]:
                l[i-1] -= 1
                print(l)
                for j in range(i,n):
                    l[j] = 9

        # Convert the List back to a number
        return int("".join([str(x) for x in l]))
obj=Solution()
obj.monotoneIncreasingDigits(55627)

```

```

[5, 5, 6, 2, 7]
[5, 5, 5, 2, 7]

```

Out[12]:

55599

In [15]:

```
class Solution:
    def monotoneIncreasingDigits(self,n):
        if (n<10):
            return n
        org=list(map(int,str(n)))
        l=len(org)
        for i in range(l-1,0,-1):
            if(org[i]<org[i-1]):
                org[i-1]-=1
                for j in range(i,l):
                    org[j]=9
        temp=int(''.join(map(str,org)))
        return temp
obj=Solution()
obj.monotoneIncreasingDigits(332)
```

Out[15]:

299

In [29]:

```

class Solution:
    def dailyTemperatures(self, temperatures):
        l=len(temperatures)
        org=[]
        for i in range(l):
            c=-1
            start=i
            while(start<l):
                print(temperatures[start],temperatures[i])
                if(temperatures[start]>temperatures[i]):
                    c+=1
                    start=l
                    org+= [c]
                    print(org)
                else:
                    print('--',temperatures[start],temperatures[i])
                    c+=1
                    start+=1
            if(c==0):
                org+= [0]
        return org
obj=Solution()
print(obj.dailyTemperatures([73,74,75,71,69,72,76,73]))
#obj.dailyTemperatures([30,40,50,60])

```

```

73 73
-- 73 73
74 73
[1]
74 74
-- 74 74
75 74
[1, 1]
75 75
-- 75 75
71 75
-- 71 75
69 75
-- 69 75
72 75
-- 72 75
76 75
[1, 1, 4]
71 71
-- 71 71
69 71
-- 69 71
72 71
[1, 1, 4, 2]
69 69
-- 69 69
72 69
[1, 1, 4, 2, 1]
72 72
-- 72 72
76 72
[1, 1, 4, 2, 1, 1]
76 76
-- 76 76

```

```
73 76
-- 73 76
73 73
-- 73 73
[1, 1, 4, 2, 1, 1, 0]
```

In [8]:

```
class Problem:
    def GCD(self,m,n):
        small=m
        if m>n:
            small=n
        for i in range(small,0,-1):
            if(m%i==0 and n%i==0):
                return i
obj=Problem()
obj.GCD(2,4)
```

Out[8]:

2

In [7]:

```
class Solution:
    def isValid(self, s):
        stack=[]
        ele=''
        for i in s:
            if i in '([{':
                stack.append(i)
            if i in ')]}':
                ele=stack.pop()
                if((i=='(' and ele=='(') or (i==']' and ele=='[') or (i=='}' and ele=='{')):
                    continue
                else:
                    return False
            if(len(stack)>0):
                return False
        return True
obj=Solution()
obj.isValid("()")
```

Out[7]:

True

In [13]:

```

class Solution:
    def canTransform(self, start, end):
        l=len(start)
        for j in range(l+1):
            for i in range(l-1):
                if (start[i:i+2]=='RX' and end[i:i+2]=='XR'):
                    start=start[:i]+'XR'+start[i+2:]
                    print(i,start,' ',end)
                elif (start[i:i+2]=='XL' and end[i:i+2]=='LX'):
                    start=start[:i]+'LX'+start[i+2:]
                    print(i,start,' ',end)
            if (start==end):
                return True
        return False
obj=Solution()
obj.canTransform("XXXXXLXXXX", "LXXXXXXXXX")

```

Out[13]:

False

In [3]:

```

s='RXXLRXXRL'
s=s[:0]+'XR'+s[2:]
s

```

Out[3]:

'XRXLRRXXL'

In [19]:

```

from fractions import Fraction
class Solution:
    def kthSmallestPrimeFraction(self, arr, k):
        org=[]
        l=len(arr)
        for i in range(l-1):
            for j in range(i+1,l):
                org+=[arr[i]/arr[j]]
        org.sort()
        org=str(Fraction(str(org[k-1])).limit_denominator()).split('/')
        print(org)
        return [int(org[0]),int(org[1])]
obj=Solution()
obj.kthSmallestPrimeFraction([1,2,3,5],3)

```

['2', '5']

Out[19]:

[2, 5]

In [24]:

```
class Solution:
    def rotatedDigits(self, n):
        good_set={'2','5','6','9'}
        count=0
        for i in range(1,n+1):
            num_set=set(str(i))
            print(num_set)
            print(good_set.isdisjoint(num_set))
            if (good_set.isdisjoint(num_set)):
                count+=1
        return count
obj=Solution()
obj.rotatedDigits(10)
```

```
{'1'}
True
{'2'}
False
{'3'}
True
{'4'}
True
{'5'}
False
{'6'}
False
{'7'}
True
{'8'}
True
{'9'}
False
{'1', '0'}
True
```

Out[24]:

6

In [26]:

```
c=0
for i in range(1,10):
    if(i>5):
        c+=1
print(c)
```

4

In [9]:

```
from collections import Counter
class Solution:
    def customSortString(self, order, s):
        o_dict=Counter(order)
        s_dict=Counter(s)
        org=""
        for i in order:
            if i in s_dict:
                org+=(i*s_dict[i])
        for i in s:
            if i not in o_dict:
                org+=i
        return org
obj=Solution()
obj.customSortString("kqep","pekeq")
```

Out[9]:

'kqEEP'

In [12]:

```

class Solution:
    def numSubarrayBoundedMax(self, nums, left, right):
        l=len(nums)
        c=0
        for i in range(l):
            start=i
            maxi=-1
            org=[]
            while(start<l):
                if(nums[start]>maxi):
                    maxi=nums[start]
                if(maxi>=left and maxi<=right):
                    org+=nums[start]
                    print(org)
                    c+=1
                    start+=1
                else:
                    start=1
            return c
obj=Solution()
obj.numSubarrayBoundedMax([73,55,36,5,55,14,9,7,72,52],32,69)

```

```

[55]
[55, 36]
[55, 36, 5]
[55, 36, 5, 55]
[55, 36, 5, 55, 14]
[55, 36, 5, 55, 14, 9]
[55, 36, 5, 55, 14, 9, 7]
[36]
[36, 5]
[36, 5, 55]
[36, 5, 55, 14]
[36, 5, 55, 14, 9]
[36, 5, 55, 14, 9, 7]
[55]
[55, 14]
[55, 14, 9]
[55, 14, 9, 7]
[52]

```

Out[12]:

18

In [17]:

```

class Solution:
    def __init__(self,nums):
        for i in range(1,nums+1):
            print(' '*i*(nums-i),int(str(1)*i)*int(str(1)*i))
obj=Solution(4)

```

```

1
121
12321
1234321

```


In [12]:

```
1*1
```

Out[12]:

1

In [13]:

```
'1'*2
```

Out[13]:

'11'

In [8]:

```
class Solution:
    def equalPairs(self, grid):
        l=len(grid)
        s=[]
        val=0
        for i in grid:
            if i not in s:
                s+=[i]
            else:
                val+=1
        c=0
        for i in range(l):
            col=[]
            for j in range(l):
                col+=grid[j][i]
            print(col,grid[0],end=","")
            if col in s:
                s+=grid[i]
                c+=1
            else:
                s+=grid[i],col]
        return c+val
obj=Solution()
obj.equalPairs([[13,13],[13,13]])
```

[13, 13] [13, 13],[13, 13] [13, 13],

Out[8]:

3

In [17]:

```

class Solution:
    def kWeakestRows(self, mat,k):
        l=len(mat)
        dic={}
        for i in range(l):
            val=mat[i].count(1)
            dic[i]=val
        dic=dict(sorted(dic.items(),key=lambda y:y[1]))
        print(dic)
        v=list(dic.keys())
        return v[:k-1]
obj=Solution()
obj.kWeakestRows([[1,1,0,0,0],[1,1,1,1,0],[1,0,0,0,0],[1,1,0,0,0],[1,1,1,1,1]],3)

```

{2: 1, 0: 2, 3: 2, 1: 4, 4: 5}

Out[17]:

[2, 0]

In [34]:

```

class Solution:
    def dailyTemperatures(self, temperatures):
        l=len(temperatures)
        org=[]
        for i in range(l-1):
            print(temperatures[i],end=" ")
            c=0
            start=i+1
            while start<l:
                if temperatures[start]<temperatures[i]:
                    c+=1
                else:
                    org+= [c]
                    start=l
                start+=1
            org+= [0]
        return org
obj=Solution()
obj.dailyTemperatures([73,74,75,71,69,72,76,73])

```

73 74 75 71 69 72 76

Out[34]:

[0, 0, 3, 1, 0, 0, 0]

In [24]:

```

class Solution:
    def findColumnWidth(self, grid):
        l=len(grid)
        org=[]
        for i in range(len(grid[0])):
            col=[]
            for j in range(l):
                col+=str(grid[j][i])
            org+=sorted(col,key=len,reverse=True)
        res=[]
        for i in org:
            res+=len(i[0])
        return res
obj=Solution()
obj.findColumnWidth([[1],[22],[333]])

```

Out[24]:

[3]

In [28]:

```

class Solution:
    def isFascinating(self, n: int) -> bool:
        twice=2*n    # double the result
        thrice=3*n    # triple the result
        concat=str(twice)+str(thrice)    # convert the twice and thrice into strings and co
        return len(concat)==len(set(concat))    # set contains no duplicates , so if leng
obj=Solution()
obj.isFascinating(267)

```

Out[28]:

True

In [31]:

```

s="ABFCACDB"
for i in range(len(s)//2):
    s=s.replace('AB','')
    s=s.replace('CD','')
s

```

Out[31]:

'FC'

In [40]:

```
class Solution:
    def maxDivScore(self, nums, divisors):
        score=0
        val=0
        for i in divisors:
            count=0
            for j in nums:
                if j%i==0:
                    count+=1
            if count>score:
                score=count
                val=i
            elif count==score:
                score=count
                val=min(val,i)
        return val
obj=Solution()
obj.maxDivScore([20,14,21,10],[5,7,5])
```

Out[40]:

5

In [1]:

```
class Solution:
    def diagonalPrime(self, nums):
        l=len(nums)
        primes=[]
        for i in range(l):
            for j in range(l):
                if (i==j or (i+j+1)==l):
                    primes+=nums[i][j]
        return primes
obj=Solution()
obj.diagonalPrime([[1,2,3],[5,6,7],[9,10,11]])
```

Out[1]:

[1, 3, 6, 9, 11]

In [5]:

```
class Solution:
    def kSmallestPairs(self, nums1, nums2, k):
        org=[]
        for i in nums1:
            for j in nums2:
                org+=[[i,j]]
        temp=sorted(org,key=lambda x:x[0]+x[1])
        return temp[:k]
obj=Solution()
obj.kSmallestPairs([1,1,2],[1,2,3],2)
```

Out[5]:

[[1, 1], [1, 1]]

In [1]:

```
org=[[1,2,1,0],[2,5,1,0],[1,1,1,1],[1,-1,0,-1]]
sorted(org,key=lambda x:x[0]+x[1]+x[2]+x[3])
```

Out[1]:

```
[[1, -1, 0, -1], [1, 2, 1, 0], [1, 1, 1, 1], [2, 5, 1, 0]]
```

In [8]:

```
class num:
    i=1
    def v1(self):
        print(self.i,end=" ")
        self.i+=1
    def v2(self):
        self.v1()
        self.v1()
        self.v1()
        self.v1()
        self.v1()
    def v3(self):
        self.v2()
        self.v2()
        self.v2()
        self.v2()
        self.v2()
obj=num()
obj.v3()
obj.v3()
obj.v3()
obj.v3()
```

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 2
9 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 8
0 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100
```

In [11]:

```
import collections as c
class Solution:
    def deleteAndEarn(self, nums):
        count=c.Counter(nums)
        nums=sorted(list(set(nums)))
        print(count)
        a,b=0,0
        i=0
        while(i<len(nums)):
            a=nums[i]*count[nums[i]]
            print(a)
            if i==0 and nums[i]==nums[i+1]-1:
                b=nums[i+1]*count[nums[i+1]]
                b=max(a,b)
                print(b)
                i+=2
            else:
                a=nums[i]*count[nums[i]]
                if nums[i]==nums[i-1]+1:
                    b=max(a,b)
                else:
                    b=a+b
                i+=1
        return b
obj=Solution()
obj.deleteAndEarn([3,4,2])
```

Counter({3: 1, 4: 1, 2: 1})

2
3
4

Out[11]:

4

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: